Internal

HOCHSCHULE FURTWANGEN UNIVERSITY | HFU

Master Thesis

Smart Exoskeleton with ML-based Analytics for Safety Applications

Submitted by: Ankitha Suma Deshpande

DOB: 30.03.1999

Matriculation Number: 272964

Fakultät Mechanical and Medical Engineering

Studiengang: Biomedical Engineering

Vorgelegt am:	30.09.2024
Semester:	SS 22/24
Gutachter 1:	Dr.Prof.Massimo Kubon (Hochschule Furtwangen)
Gutachter 2:	Dr. Oliver Harnack (Dräger Safety AG & Co KGaA)

ACKNOWLEDGMENT

The opportunity to write my master's thesis with Dräger Safety AG & Co KGaA was a pivotal moment for my personal and professional growth. I am deeply grateful for the privilege of learning from industry leaders and wish to extend my heartfelt appreciation to everyone who supported me along the way.

I am grateful to Dr. Oliver Harnack, Peer Gross, and Kaden Holger, whose unwavering support and invaluable guidance were instrumental throughout the project. Their enthusiastic encouragement and insightful feedback were truly invaluable.

I also want to express my sincere appreciation to Prof. Dr. Massimo Kubon for his supervision from the university and his genuine interest in the project's success.

Lastly, I am thankful to all those who, directly or indirectly, contributed to this endeavor. Your assistance has made a significant difference, and I am profoundly grateful for your support.

EIDESSTATTLICHE ERKLÄRUNG

Ich erkläre hiermit, dass ich diese Thesis selbstständig ohne Hilfe Dritter und ohne Benutzung anderer als der angegebenen Quellen und Hilfsmittel verfasst habe. Alle den benutzten Quellen wörtlich oder sinngemäß entnommenen Stellen sind als solche entsprechend kenntlich gemacht und aufgelistet.

Diese Arbeit ist bislang keiner anderen Prüfungsbehörde vorgelegt worden und auch nicht veröffentlicht worden.

Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben kann.

Lübeck, 30.09.2024

Ankitha

Ort, Datum, Unterschrift

Contents

ACKNOWLEDGMENT ---- 3
EIDESSTATTLICHE ERKLÄRUNG ---- 4
Table of Figures ---- 7
List of Tables ---- 8
1 INTRODUCTION ---- 10
1.1 Motivation ---- 10
1.2 Background ---- 11
1.3 Problem Statement ---- 12
1.4 Structure of the report ---- 12
1.5 Hypotheses ---- 13
2 TECHNICAL BACKGROUND ---- 15
2.1 Exoskeleton ---- 15
2.1.1 Active Exoskeleton ---- 16
2.1.2 Passive Exoskeleton ---- 17
2.1.3 3D printing ---- 18
2.2 Machine Learning ---- 19
2.2.1 Types of Machine Learning ---- 20
2.2.2 Decision Tree ---- 21
2.2.3 Overfitting and Underfitting ---- 21
2.3 MQTT ---- 22
2.4 ThingsBoard ---- 23
2.5 NTP ---- 24
2.6 Raspberry Pi Pico W ---- 24
2.7 MPU6050 ---- 25
3 CONCEPT ---- 26
3.1 Analysis of the situation ---- 26
3.1.1 Static Positions ---- 26
3.1.2 Dynamic positions ---- 26
3.2 Requirement for further development ---- 27
3.3 Software Components ---- 28
3.3.1 Mechanical Setup ---- 29
3.3.2 Experimental Setup ---- 31

3.3.3 Principle scheme ---- 35
3.3.4 Data Preparation ---- 36
4 IMPLEMENTATION ---- 39
4.1 Data Generation ---- 39
4.2 Training the exoskeleton model ---- 43
4.3 Live prediction ---- 44
5 CHALLENGES AND MITIGATION STRATEGIES ---- 46
5.1 Data Emission Signal ---- 46
5.2 Data Loss ---- 46
5.3 Lost Pico Synchronization ---- 47
5.4 Classification Model ---- 48
6 RESULTS ---- 49
6.1 Static Data analysis ---- 49
6.2 Dynamic data analysis ---- 52
7 CONCLUSION AND OUTLOOK ---- 57
7.1 Conclusion ---- 57
7.2 Outlook ---- 58
8 REFERENCE ---- 59
9 APPENDIX ---- 62

Table of Figures

Figure 1 – Visual representation of the exoskeleton demonstrator used in the experiment. ----------16
Figure 2 - Powered tools used in an active exoskeleton structure for functionality [9].----------17
Figure 3 - Mechanical structure used in the passive exoskeleton design [9]. ----------17
Figure 4 - Classification of Machine Learning [15]. ----------20
Figure 5 - Decision tree structure used for machine learning classification [15]. ----------21
Figure 6 - Comparison of overfitting and underfitting in machine learning models [20]. ----------22
Figure 7 - IoT communication process using MQTT protocol for data exchange [21]. ----------23
Figure 8 - The basic ThingsBoard architecture [26].----------24
Figure 9 - Raspberry Pi Pico W microcontroller used for sensor data processing [30]. ----------25
Figure 10 - MPU6050 sensor used for motion detection [33].----------25
Figure 11 – Initial sensor analysis before the calibration process.----------27
Figure 12 - Sensor analysis post-calibration showing improved accuracy.----------28
Figure 13 - Software Components used in this project.----------29
Figure 14 – Process flow of individual components in the exoskeleton system. ----------29
Figure 15 - FDM method for 3D printing exoskeleton.----------30
Figure 16 - SLS method for 3D printing exoskeleton.----------30
Figure 17 - Exoskeleton with Airboss for enhanced functionality. ----------31
Figure 18 - Exact placements of 9 sensors on the exoskeleton (color coordinated)----------32
Figure 19 – Flowchart outlining the experiment process for exoskeleton testing.----------34
Figure 20 – Flowchart showcasing the software process involved in the exoskeleton system. ----------35
Figure 21 - Data Flow and System Architecture for Real-time Prediction in the Exoskeleton Setup-------36
Figure 22 - Forward Bending (Oberkoerper vor)----------37
Figure 23 - Standing (Stehen)----------37
Figure 24 - Balance on left knee (knie rechts vor)----------38
Figure 25 - Balance on right knee (knie links vor) ----------38
Figure 26 - Sleep on stomach (liegen bauch)----------38
Figure 27 - Accelerometer calibration process to ensure accurate motion tracking [35]. ----------39
Figure 28 - Gyroscope calibration part 1: assessing rotational data accuracy [35]. ----------40
Figure 29 - Gyroscope calibration part 2: intermediate stage of data adjustment [35].----------40
Figure 30 - Gyroscope calibration part 3: final calibration step for precise tracking [35].----------40
Figure 31 - Process of recording sensor data into a text file for analysis. ----------41
Figure 32 - Sorting collected data to facilitate easier analysis and visualization. ----------42
Figure 33 - Parsed data displayed after processing for clearer interpretation. ----------43
Figure 34 - Potential Location for Data Loss in the System. ----------47
Figure 35 - Implementation of NTP in the system.----------48
Figure 36 - Data flow diagram for live prediction of movement using sensors.----------49
Figure 37 - Transition between static positions during movement detection analysis. ----------51
Figure 38 - Live detection of body positions captured by the exoskeleton sensors. ----------51
Figure 39 - Fast walking movement frequency spectrum (FFT analysis).----------53
Figure 40 - Time-domain representation of fast walking movement data. ----------54
Figure 41 - Slow walking movement frequency spectrum (FFT analysis).----------55
Figure 42 - Time-domain representation of slow walking movement data. ----------56

List of Tables

Table 1 - Placement and measurement description of the 9 sensors on the exoskeleton. 33
Table 2 – Live prediction performance. 50

1 INTRODUCTION

Smart exoskeletons equipped with machine learning (ML) are a promising solution to increase safety in various applications. These exoskeletons, equipped with sensors and actuators, provide physical support and assistance to the user, especially during demanding tasks or in industrial environments. Using ML algorithms, the exoskeleton can improve human strength and endurance, reduce the risk of injury, and increase productivity in physically demanding tasks.

Machine learning algorithms embedded in these exoskeletons assess real-time data inputs such as user movements, environmental circumstances, and task specifications. By continuously processing this data, the exoskeleton can alter its support levels and provide individualized assistance to the user, enhancing both safety and performance. [1] [2].

This innovative approach to exoskeleton technology can potentially revolutionize safety in manufacturing, construction, healthcare, and logistics industries. Through the use of machine learning, these smart exoskeletons can detect potential safety risks, predict user fatigue or strain, and provide timely warnings or adjustments to prevent accidents [2]

ML algorithms are crucial in processing sensor data and predicting user movements so that smart exoskeletons can dynamically adapt to different tasks and environments. In industry, for example, ML-based analytics can recognize worker movement patterns and provide timely feedback to prevent ergonomic hazards or repetitive strain injuries (RSI) [3].

Integrating ML-based analytics into smart exoskeletons represents a significant advance in safety technology and offers potential benefits in various sectors, including manufacturing, healthcare, and construction. However, further research is needed to optimize the algorithms, improve sensor accuracy, and ensure seamless human-machine interaction to achieve widespread adoption and maximum effectiveness.

1.1 Motivation

Choosing the topic "Smart Exoskeleton with ML-based Analytics for Safety Applications" for my master’s thesis presents an exciting opportunity to work with cutting-edge technologies that have broad implications for safety across multiple fields. By blending wearable robotics, machine learning, and safety engineering, this research tackles a multidisciplinary challenge with real-world relevance.

Smart exoskeletons, while commonly associated with industrial safety and firefighting, have diverse applications. In healthcare, exoskeletons are transforming rehabilitation by helping patients with mobility impairments regain movement, offering personalized support for individuals recovering from strokes or spinal injuries. They are also being used in elderly care to maintain independence.

In industrial settings, exoskeletons are already improving workplace safety by reducing strain from repetitive tasks like heavy lifting. Companies are adopting them to prevent injuries and increase productivity, allowing workers to perform demanding tasks for longer periods with less fatigue. Similarly, in the military, exoskeletons help soldiers carry heavy loads, enhancing endurance and agility in challenging conditions.

Agriculture is another sector where exoskeletons are making an impact. By reducing the physical strain of tasks like bending and lifting, these devices are making farm work safer and more efficient.

The integration of machine learning adds a layer of intelligence to these systems, enabling real-time analysis of sensor data to provide adaptive, personalized support based on the user's specific movements. In this way, exoskeletons become more than just mechanical aids—they actively adjust to the needs of the user, enhancing both performance and safety.

By exploring and developing ML algorithms for smart exoskeletons, this research aims to push the boundaries of personalized assistance and proactive safety. The potential applications extend far beyond a single industry, with the opportunity to improve workplace safety, enhance rehabilitation, and elevate quality of life for individuals with mobility challenges. This thesis, therefore, offers a compelling mix of technological innovation and practical impact, contributing to the advancement of safety and assistive technology in meaningful ways.

1.2 Background

Dräger Safety AG & Co KGaA is an international company based in Germany that specializes in medical and safety technology. Johann Heinrich Dräger founded the company in 1889 in Lübeck and initially focused on manufacturing equipment for shipping and medical applications. Over the years, Dräger has expanded its product portfolio and global presence to become a leading provider of innovative solutions for various industries.

In the medical sector, Dräger is known for its wide range of medical devices and systems used in anesthesia, ventilation, patient monitoring, and neonatal care. The company's medical devices meet the highest quality, safety, and reliability standards, making them a trusted choice for healthcare professionals worldwide.

In addition to its medical division, Dräger is a leading provider of personal and occupational safety products and services in safety technology. These include respiratory protection, gas detection, and firefighting equipment, among others. Dräger's safety solutions are tailored to the unique needs of industries such as oil and gas, chemicals, pharmaceuticals, and firefighting.

Dräger has earned a reputation as a global medical and safety technology leader due to its commitment to innovation and excellence. With a strong focus on research and development, the company continues to bring new products and solutions to the market that improve people's and organizations' safety, efficiency, and quality of life worldwide.

1.3 Problem Statement

The existing prototype of an exoskeleton aims to optimize and extend its functionality by integrating digital signal processing and machine learning algorithms. This exoskeleton improves the body's stability, strength, and mobility and monitors real-time vital data. The main goal is to enhance the wearable device's capabilities beyond physical assistance to include real-time biometric pattern recognition to provide biomedical advice and trigger alarms in critical situations. The research aims at safety, firefighting, and industrial sector applications.

Key components of the exoskeleton system include gyro sensors to capture motion signals and sensors to monitor vital body data such as heart rate, blood pressure, and oxygen saturation. Real-time processing and analysis of this data through ML algorithms enables the identification of specific patterns that indicate the wearer's physical and health condition. By distinguishing between every day and potentially hazardous situations, the system can make timely suggestions to promote healthier behaviors and prevent injury.

A crucial aspect of this study is developing a pattern recognition model tailored to detect abnormal physiological and movement patterns. This requires extensive data collection to train the model, overcome challenges such as data variability, and develop robust algorithms capable of learning from complex, multidimensional data.

The practical implications of this research extend to various areas such as safety, firefighting, rehabilitation, health, and elderly care. Through detailed system design, implementation, and testing analysis, this study contributes valuable insights into integrating wearable technology, digital signal processing, and ML to improve human health and performance.

This research aims to demonstrate the feasibility and effectiveness of using an ML-enabled exoskeleton to improve quality of life through advanced biomechanical support and health monitoring. By laying the foundation for future advances in wearable technology, it provides a blueprint for developing smart, adaptive, and personalized healthcare solutions.

1.4 Structure of the report

This report consists of seven chapters. Chapter One states the introduction and intention of the master thesis. The core contents of the work are stated from chapter two to chapter six. Chapter two consists of the technical background skills that the project requires and the software and hardware concepts involved in the techniques that will be used further.

Chapter three dives into analyzing the situation, developing the concept of the project, and determining the initial steps to take. Setup of the software and hardware components and the requirements that need to be set up for further development. Chapter four is all about the detailed implementation and the strategies used. Chapter five speaks about the challenges that were faced

during the project and the ways adapted to mitigate them. Chapter six gives details about the result of the project, and chapter seven gives the overall conclusion and outlook of the project.

Finally, chapter eight lists all the references used in the report, and chapter nine is the appendix, which has codes and all the additional information needed.

1.5 Hypotheses

The primary goal of this thesis is to improve the exoskeleton's ability to support physical movements while monitoring the wearer's vital health data in real-time. More specifically, the focus is on enhancing the exoskeleton's capacity to predict and adapt to dynamic movements. The research aims to test the following hypotheses:

- Improved Physical Support: We hypothesize that incorporating machine learning algorithms will significantly enhance the exoskeleton’s ability to support the body in terms of stability, strength, and mobility, particularly during dynamic tasks.
- Accurate Prediction of Movements: It is hypothesized that the integration of gyroscopic sensors and machine learning models will allow the exoskeleton to accurately predict and respond to a variety of dynamic movements, such as walking, bending, and climbing, in real-time with minimal latency.
- Real-time Health Monitoring: We hypothesize that the system will effectively detect abnormal biometric patterns, using ML to analyze physiological and movement data, and deliver real-time feedback or alerts to enhance user safety.
- Practical Application in Firefighting and Safety: The exoskeleton is expected to demonstrate clear benefits in firefighting, showing improved physical support and safety monitoring in high-risk environments.

The key aspects of the research involve:

- Using gyro sensors to capture motion signals.
- Processing and analyzing data in real-time through ML algorithms.
- Collecting large data sets to train the pattern recognition model.
- Overcoming challenges related to data variability and algorithm robustness.

The research's practical implications extend to security, firefighting, rehabilitation, healthcare, and elderly care, but this project primarily focuses on the fire department. Through system design, implementation, and testing, this work aims to provide valuable insights into integrating wearable technology, digital signal processing, and machine learning to improve human performance and health monitoring.

This thesis ultimately aims to demonstrate the feasibility and effectiveness of using an ML-powered exoskeleton to improve quality of life through advanced biomechanical support and real-time health monitoring. The findings will lay a foundation for future advancements in wearable technology and personalized healthcare solutions.

2 TECHNICAL BACKGROUND

This chapter explains the technical aspects fundamentally to help readers understand how to implement the thesis further. It briefly describes the exoskeleton, machine learning, Raspberry Pi sensor, MPU 6050, and MQTT.

2.1 Exoskeleton

The external skeleton supports the body and provides strength, stability, and mobility. It is used ergonomically to support the body, promote movement, or provide compatible support during rehabilitation [4]. Advances in actuators, sensors, batteries, technology, and biocompatibility have raised hopes for exoskeletons that enable people to live better lives [4]. There are a multitude of possible applications for the exoskeleton [4]. Patients on their joints or limbs wear the exoskeleton, which helps them to perform better in their physiotherapy training [5] and improve their quality of life in everyday life.

There are two variants for an exoskeleton – user-grounded and externally grounded. An external user or structure does not support user-grounded exoskeletons [6]. The user can use the workspace indefinitely, but unfortunately, it has power restrictions because a device stores the energy, or a power cable is connected that restricts movement [4] [6]. A grounded exoskeleton, on the other hand, can be worn outside the clinical environment and supports the wearer in their daily activities [6]. On the other hand, grounded exoskeletons are supported externally by being attached to the wearer and a stationary reference such as a wall or floor. This also restricts the working area as the exoskeleton can only move from the grounded interface [6].

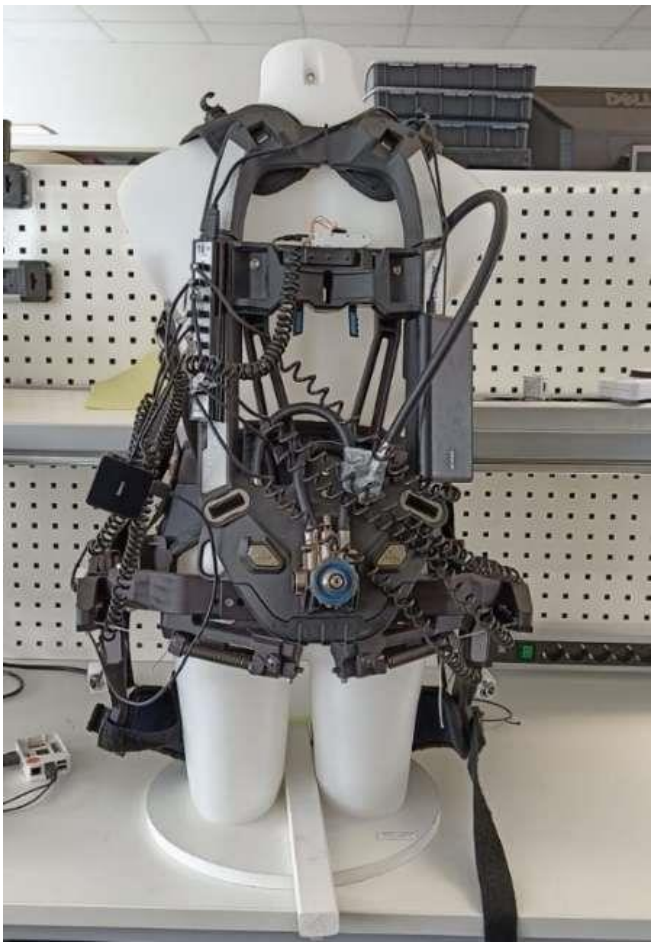


Figure 1 – Visual representation of the exoskeleton demonstrator used in the experiment.

Researchers have separated exoskeletons into two categories: active and passive.

2.1.1 Active Exoskeleton

At least one actuator, coupled to the body and designed to mimic the human anatomy, is a feature of an active exoskeleton [7]. Active industrial exoskeletons have remained in research and development, unlike passive exoskeletons, where a few are in the market [8]. These have greater sophistication and complexity. When not needed, active or powered exoskeletons can move freely and with flexibility thanks to a combination of batteries, sensor-based controllers, and actuators that adjust buoyancy [9]. Powered tools Figure 2 distribute the workload by producing a positive force that lessens the total burden of recurrent lifting activities and frees up the user's energy to concentrate on finishing the current activity [9].

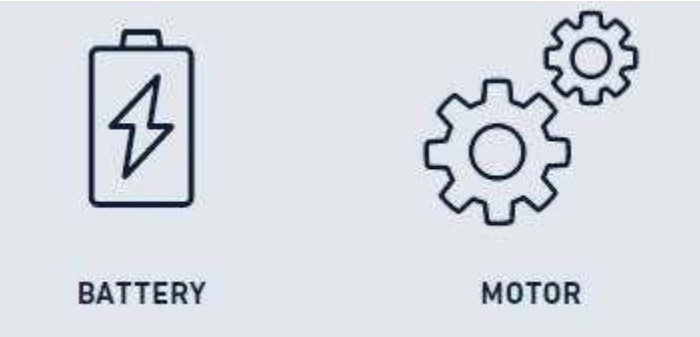


Figure 2 - Powered tools used in an active exoskeleton structure for functionality [9].

2.1.2 Passive Exoskeleton

Passive exoskeletons redistribute forces from physically demanding tasks through material and structural features [7]. By employing the restoring force of springs and dampers Figure 3, these entirely mechanical structures provide support for the user during lifting motions [9]. These exoskeletons focus on reducing tension from the lower back and have succeeded in dynamic lifting and static holding activities.

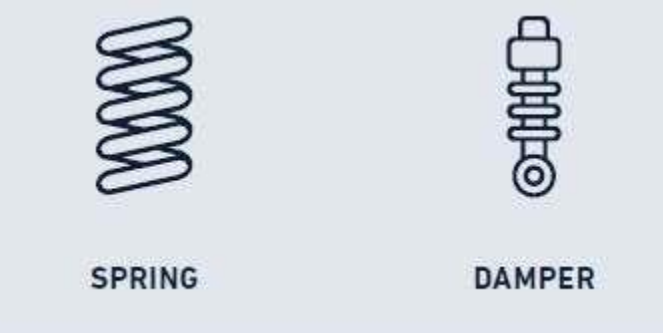


Figure 3 - Mechanical structure used in the passive exoskeleton design [9].

Additionally, this has a few disadvantages. Although it lessens the strain on the lower back, other muscles may become more active depending on the activity. The leg muscles exhibit the most significant increase in activity [10]. There is also more leg activity and oxygen intake during extended lifting and lowering tasks. Discomfort in confined places, such as the knees and shoulders, may also be an issue. For exoskeletons to work, pressure must be applied to the body [10]. These contact areas could become uncomfortable or even injured if not constructed with care, making users reluctant to wear the exoskeleton [10].

2.1.3 3D printing

Materials are joined to create products from 3D model data using 3D printing, often referred to as additive manufacturing (AM), rapid prototyping (RP), or solid-freeform (SFF) [11]. Intending to minimize waste and achieve acceptable geometric precision, this technique adds materials to the creation process of products [11] [12]. Building buildings or acquiring picture data in CAD software can produce a meshed 3D computer model, the first step in 3D printing. The materials currently manufactured using 3D printing technology are metal, ceramics, conventional thermoplastics, and materials based on graphene [13]. A growing number of industries, including aerospace, automotive, healthcare, and agricultural, are using this technology for mass manufacturing or customization [13] [14].

2.1.3.1 Types of 3D printing

A variety of 3D printing technologies have been developed with different functions.

- Binder jetting: A type of 3D printing in which powder particles are selectively coated with a liquid binder, enabling the production of massive, reasonably priced items out of a variety of materials, including sand and metals [13].
- Directed Energy Deposition: A sophisticated 3D printing technique with excellent control over grain structure and quality utilized mostly with metal components for component additions or repairs [13].
- Materials Extrusion: Often employed in FDM procedures, materials extrusion is a popular and affordable 3D printing technique that constructs items layer by layer by extruding hot thermoplastic [13].
- Materials Jetting: A highly accurate 3D printing technique that creates items with smooth surfaces and excellent accuracy by selectively depositing and solidifying material droplets under UV light [13].
- Powder Bed Fusion: Includes techniques like Selective Laser Sintering (SLS), which uses a high-power laser to sinter powder materials, creating accurate 3D objects in metals, plastics, and ceramics [13].
- Sheet Lamination: This 3D printing technique offers advantages, including full-color printing, recyclability, and cost-effectiveness by bonding layers of material sheets to build things [13].
- Vat Photopolymerization: A UV-curing method that uses light to cure photo-reactive polymers; widely used in SLA and DLP processes to create finely detailed, superior-quality products [13].

2.1.3.2 Materials used for 3D printing.

To produce consistently high-quality products, 3D printing requires premium materials that adhere to strict criteria.

- Metals: Because of its exceptional physical qualities, metal 3D printing is extensively utilized in the automotive, aerospace, and medical industries. It enables the creation of intricate parts from materials like titanium, aluminum, and alloys based on nickel [13].
- Polymers: Due to their low cost, lightweight nature, and adaptability in applications such as medical devices, 3D printing with polymers such as PLA and ABS is popular for producing everything from prototypes to functioning constructions [13].
- Ceramics: By employing materials like alumina, bioactive glass, and zirconia, 3D printing with ceramics creates robust, long-lasting items appropriate for dental, aerospace, and construction applications [13].
- Composites: Because of their lightweight and customizable nature, composite materials, such as glass fiber- and carbon fiber-reinforced polymers, are employed in 3D printing for high-performance applications in sectors like aerospace [13].
- Smart Materials: Shape memory alloys and polymers are examples of smart materials that can be used in 3D printing [13]. These materials have the ability to alter shape in response to external stimuli, opening up novel applications like soft robotics and self-evolving structures [13].
- Special Materials: Food, lunar dust, and textiles are examples of unique materials that can be used with 3D printing to create personalized food products, possible equipment for moon colonization, and fashion industry breakthroughs [13].

2.2 Machine Learning

In today's digital environment, a multitude of technologies, including social media, cybersecurity, data from the Internet of Things (IoT), and more, are available. Artificial intelligence, particularly the precise technique known as machine learning, is important for analyzing data intelligently and creating appropriate intelligent and automated applications [15].

Through AI, machine learning enables systems to learn independently and improve through experience without being programmed. Creating computer programs that can access data and allow the system to learn on its own is the primary goal of machine learning [15]. Reading or observing data is the first step in the learning process. Either clear instructions or practical expertise to search for patterns in the data and use the provided examples to inform forthcoming judgments.

2.2.1 Types of Machine Learning

The types of machine learning are explained briefly in Figure 4.

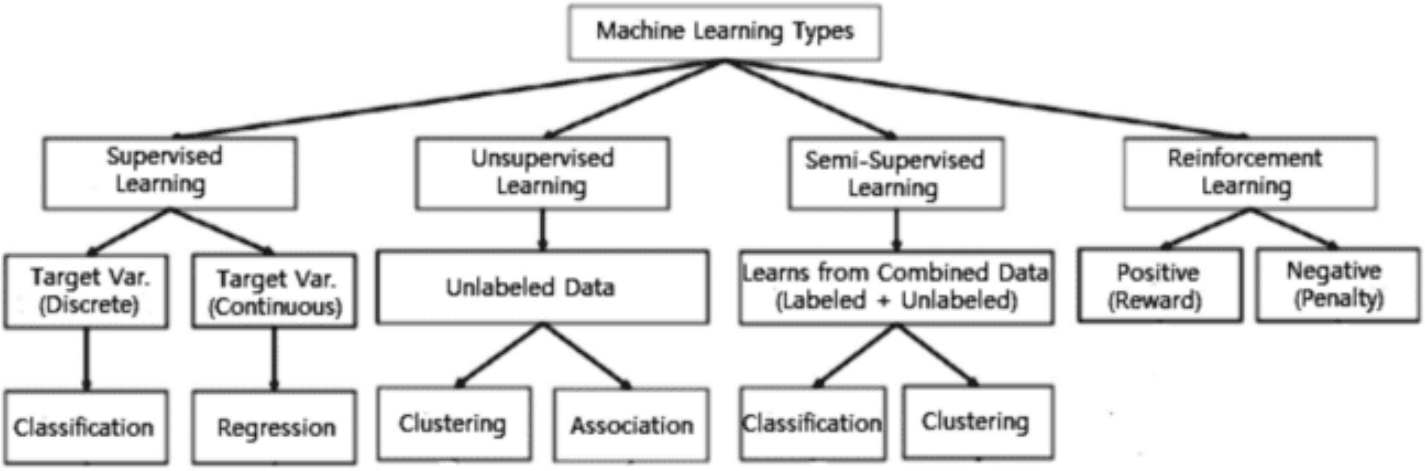


Figure 4 - Classification of Machine Learning [15].

- Supervised Learning: The simplest type of machine learning is supervised learning, where a function maps an input and an output depending on input samples [16]. When precise objectives are identified to be accomplished with a specific set of inputs, supervised learning is carried out [15]. Machine learning algorithms that need outside assistance are known as supervised algorithms [16].
- Unsupervised Learning: Unsupervised learning is the counterpart to supervised learning. During the learning process, the machine does not mark the content in a particular way [16]. The algorithms can identify and display the intriguing structures they come across in the data [17]. The method teaches the machine the fundamental ideas and information, like tree roots. The data class is identified upon introducing the raw data by referencing previously learned features [17].
- Semi-supervised Learning: This holds true for data that is labeled or unlabeled. While there is an extremely high chance of finding unlabeled data in the actual world, labeled data is incredibly rare [15]. Semi-supervised learning is utilized to get a prediction result superior to the labeled data. Fraud detection and machine translation are two instances [15].
- Reinforcement Learning is a systematic approach to learning a subject. The system uses previously examined data [16] as a reference. This kind of learning, focused on rewards and punishments, is primarily employed to lower risk or create success-rate-boosting tactics [15]. Data gathering is expanded in this kind of learning, which is based on both static and dynamic learning.

2.2.2 Decision Tree

One of the various techniques for classifying algorithms is the decision tree. This technique has several application areas and is well-known as a non-parametric supervised learning technique [15]. According to the principle of operation, data information is processed beginning at the root node and ending at the final node, where it converges to complete the process.

Figure 5 shows classifying the instances is aided by sorting the tree using a top-down approach. The decision number method divides the branches and prunes them simultaneously to enhance the data content's integrity and make the data information analysis easier [16]. The contents of two nodes are examined in this analysis to find the best qualities [18]. This gives one a thorough understanding of the division's data. Moreover, the branching method - such as a tree - can enhance the number of samples analyzed and, based on sample number statistics, identify the content with the most significant number of samples in the classification [16].

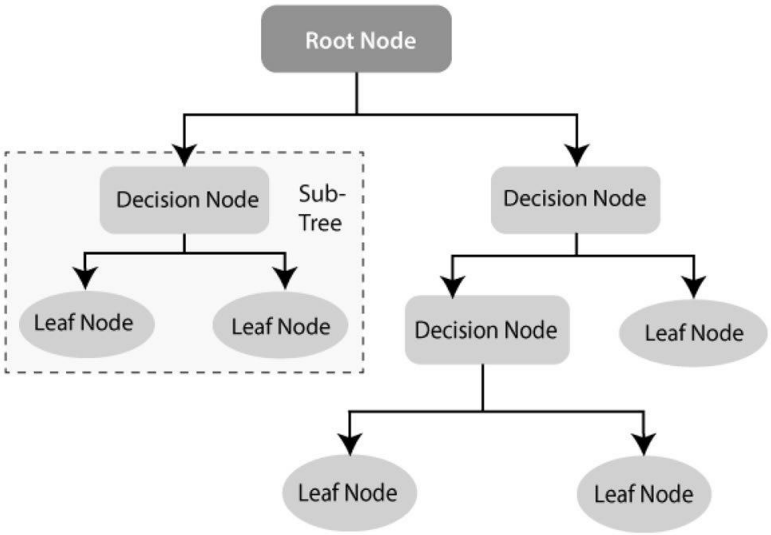


Figure 5 - Decision tree structure used for machine learning classification [15].

2.2.3 Overfitting and Underfitting

Overfitting and underfitting are two of the many issues that might arise during model training. Overfitting becomes an issue when there is an excessive difference between the test and training errors [19]. Stated otherwise, this indicates that the training data sets significantly outperform the test data sets for the model. For this approach, generalizing the unknown data is challenging [18].

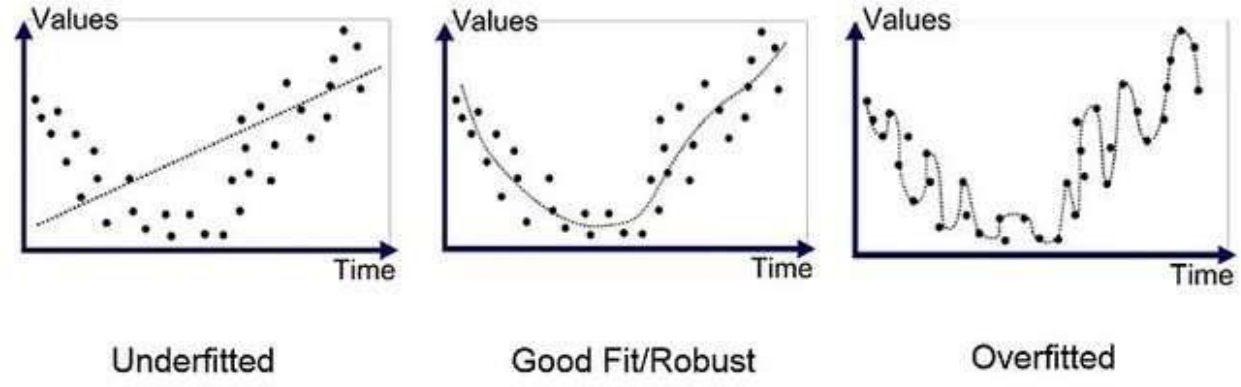


Figure 6 - Comparison of overfitting and underfitting in machine learning models [20].

When the model cannot obtain a low enough error on the training set, underfitting takes place. This can also be explained by the fact that the model cannot even assess the training data since it has not been trained with adequate data. That is why it will be challenging to obtain a nearly precise response [18].

2.3 MQTT

A vast number of smart systems use several communication protocols to communicate within the Internet of Things (IoT). A popular publish-subscribe protocol used for transmitting sensor or event data is called Message Queuing Telemetry Transport (MQTT) [21]. MQTT is best suited for small devices and networks because of its low bandwidth, high latency, and unstable networks. Among other domains, MQTT is extensively used in industrial, Internet of Things, smart home, and agricultural applications [22]. This is due to the fact that it has reduced packet loss, consumes less memory, and facilitates low-bandwidth communication [21].

The broker (central device), clients (IoT nodes), topics, and messages are the four primary parts of MQTT. Topics are essential to transmit messages across networks since they contain source and destination node information. Forward slash delimiters are used to group these subjects, and the messages contain the data gathered by IoT sensors. Subject selection, topic publication, and topic subscription are the three primary responsibilities of any node utilizing the MQTT message transport protocol. First, a central node known as the "broker" facilitates communication between the IoT nodes (clients). This broker can operate remotely in the cloud or locally on the edge. If the node's capability permits, it makes it simpler for IoT nodes to publish, subscribe to, or both at the same time Figure 7 [21].

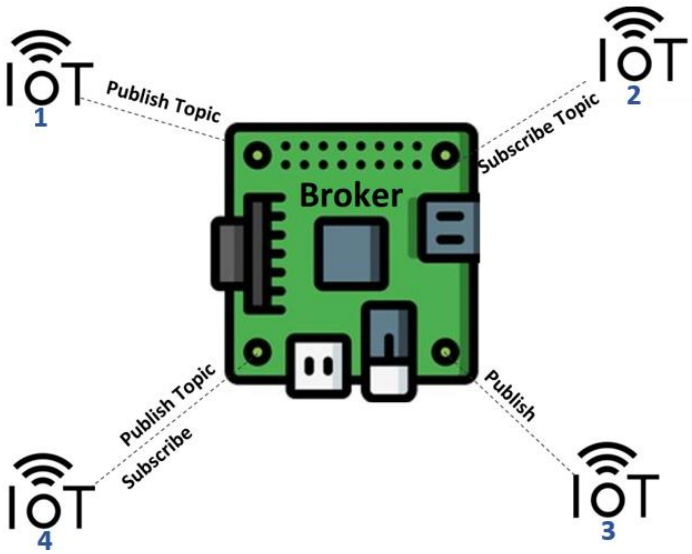


Figure 7 - IoT communication process using MQTT protocol for data exchange [21].

2.4 ThingsBoard

Thingsboard is one of the seven open-source IoT platforms. The papers that were published before June 2021 were collected for the comparison of the platforms, which resulted in the collection of 135 papers [23]. The seven candidates were compared considering 14 independent factors, and ThingsBoard was rated highest [23]. ThingsBoard is an open-source platform that allows one to collect, process, visualize, and manage data from various devices [24]. This provides a good solution for data management, data storage, analysis, data visualization, and data processing and analytics [25]. ThingsBoard supports standard IoT protocols for device connectivity, including MQTT, CoAP, and HTTP(S) [26]. To save the telemetry data (time-series sensor, readings, statistics, etc.) and entities (devices, assets, users, alarms, etc.,), the ThingsBoard uses SQL, NoSQL, and Hybrid databases [26].

The ThingsBoard architecture is designed to be scalable, fault-tolerant, robust, efficient, customizable, and durable. The basic architecture is shown in Figure 8.

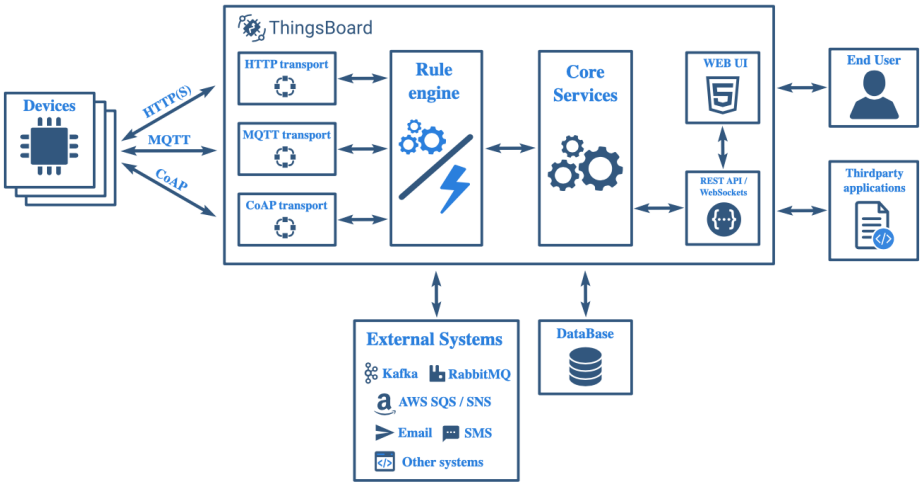


Figure 8 - The basic ThingsBoard architecture [26].

2.5 NTP

Time synchronization plays a crucial role in the modern world, contributing significantly to the management, security, planning, and debugging of networked systems [27]. Accurate time coordination is essential for ensuring reliable performance and precise analysis in projects that involve interconnected devices and data exchange [27]. The Network Time Protocol (NTP) is widely used for synchronizing the clocks of devices across networks. Many systems achieve accurate time synchronization by implementing NTP software, allowing their devices to connect with NTP servers to keep their clocks aligned [28]. A hardware clock is installed on the system or computer, but with the slightest change in temperature and crystal oscillators, no two devices will show the same time [28]. NTP is known to be the oldest and most accurate internet protocol. It has been known for its efficiency in synchronization of time across the system [28].

2.6 Raspberry Pi Pico W

A compact, inexpensive single-board computer that may be used by individuals of various backgrounds and skill levels to enjoy and learn computing is the Raspberry Pi Pico W with RP2040 microcontroller [29]. It is a programmable device without internal memory or peripherals that possesses all the critical features of a motherboard. You insert an SD card into the designated slot to configure the Raspberry Pi. The Linux operating system is compatible with the Raspberry Pi [29]. The Pico W model may be programmed using Micro Python and features an integrated Wi-Fi circuit. Also, a temperature sensor is integrated inside it [30].

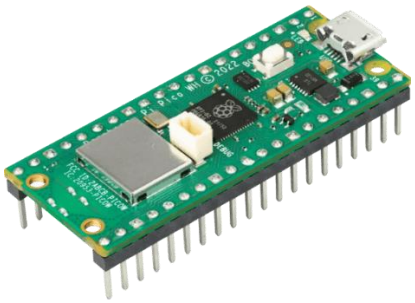


Figure 9 - Raspberry Pi Pico W microcontroller used for sensor data processing [30].

2.7 MPU6050

The MPU6050 is an IMU with six degrees of freedom that measures angular acceleration using a three-axis gyroscope. It measures linear acceleration using the remaining three axes of linear accelerometers [31]. When motion detection is necessary, it is employed in robotics, game controllers, and other electronic devices [32].

Piezoelectricity, or the capacity to produce an electrical charge in reaction to mechanical stress, is how three-axis accelerometers function. Coriolis acceleration is the basis for operating other 3-axis gyroscopes [32].

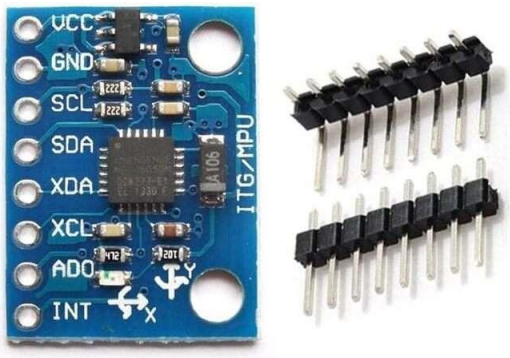


Figure 10 - MPU6050 sensor used for motion detection [33].

3 CONCEPT

The scenarios and theory of the exoskeleton's setup are covered in this chapter.

3.1 Analysis of the situation

The purpose of this prototype exoskeleton is to aid firemen. This exoskeleton relieves pressure on their backs as they perform their hard labor. This exoskeleton is an upgrade for an already existing and in-use SCBA. The oxygen tank can be attached to the rear and intended to be worn like a backpack. The Raspberry Pi Picos with MPU6050 is mounted to the exoskeleton. These sensors enable the acquisition of movement-related gyroscope data and linear acceleration. These sensors facilitate the analysis of the wearer's situation. There are nine sensors, most arranged to enhance the accuracy of readings related to thigh and pelvic motions in the pelvic region. On the back is just one sensor integrated into a pressure sensor. This pressure sensor integrated pico facilitates the investigation of the dynamic locations during stair climbing and descending, as well as the pressure differential felt when bending forward in a static posture. The exoskeleton might be equipped with a battery bank or powered by a power source to enable mobility. A spring supports this passive exoskeleton, as seen in Figure 3.

The data collection and analysis are focused on two categories of positions.

3.1.1 Static Positions

The participants had to maintain their positions at all times. The participants were required to maintain a range of postures, such as standing, sleeping on the stomach, leaning forward, and balancing on both the left and right knees. All nine sensors' worth of data were gathered at each location. A twenty-second recording was made of each position.

3.1.2 Dynamic positions

The dynamic movements that are included in this project are slow walking, fast walking, and crawling for the first stage of development. The sensor placements are as same as the static position and the participants have performed the movement for twenty seconds. The collected sensor data were analyzed which explained in detailed in the following chapter.

3.2 Requirement for further development

The exoskeleton is mechanically modeled. A properly functioning and calibrated sensor is required for meaningful data analysis. Various iterations of the sensors underwent testing. To ensure proper functionality, each sensor and Pico were examined independently. Prior to calibration, Figure 11 depicts the sensor's operational state. As can be observed, each sensor exhibits a great deal of variation.

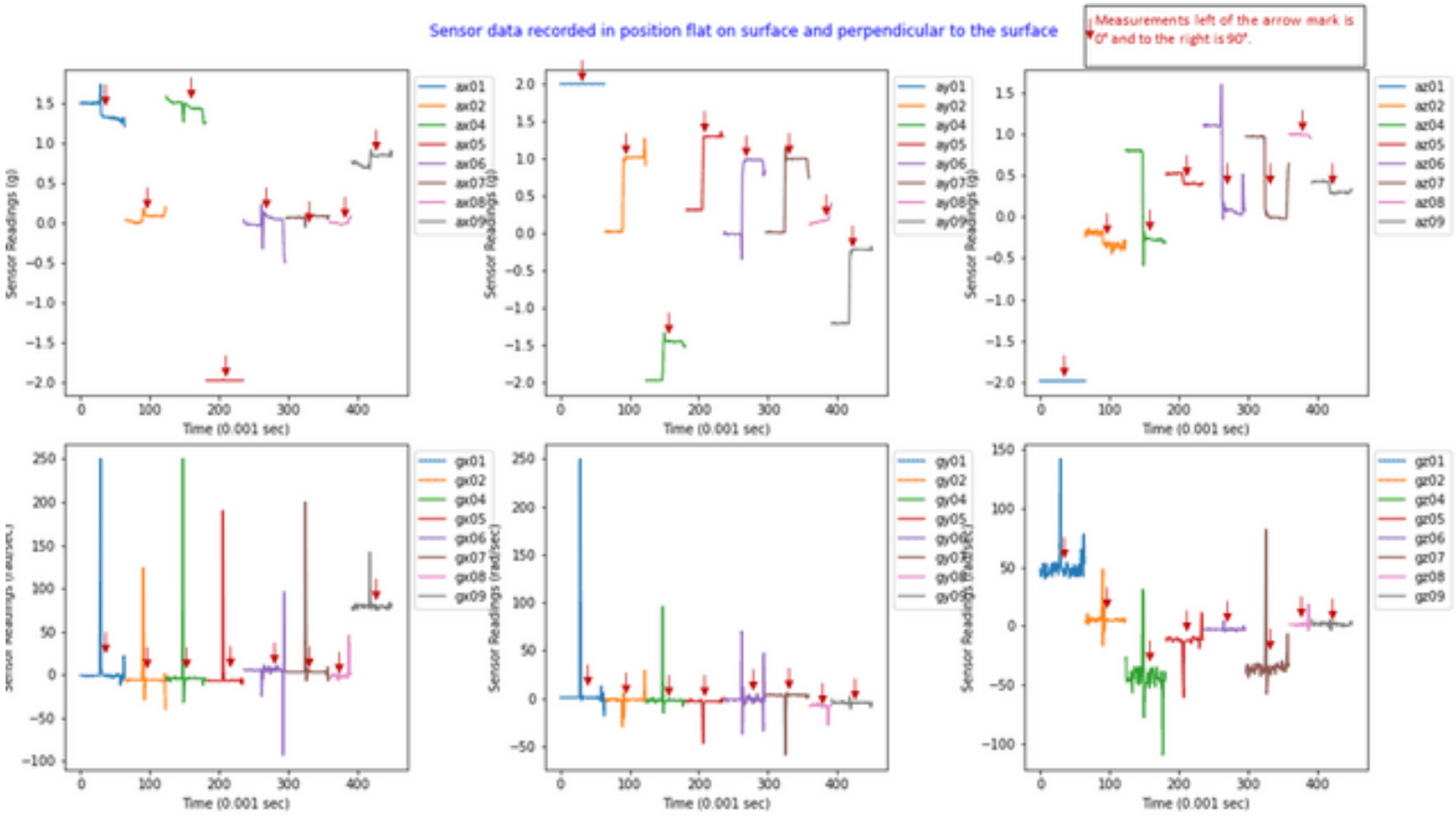


Figure 11 – Initial sensor analysis before the calibration process.

Neither the scale nor the offset is preserved. This makes it difficult to understand the situation and generates many mistakes in the recorded data. A few sensors have a fault that results in a constant value. Sensors 1 and 5 at the ax, ay, and az axes, as shown in Figure 11 above, yield a constant value, making it challenging to assess the recorded data. As seen in Figure 11, the sensor data were recorded in two different orientations: parallel to the gravitational pull, flat on the surface, or 180 degrees away from it. The second position is also 90° from the surface, perpendicular to the gravitational pull.

The sensors were calibrated after the initial study. Figure 12 illustrates how the data appears more consistent and transparent.

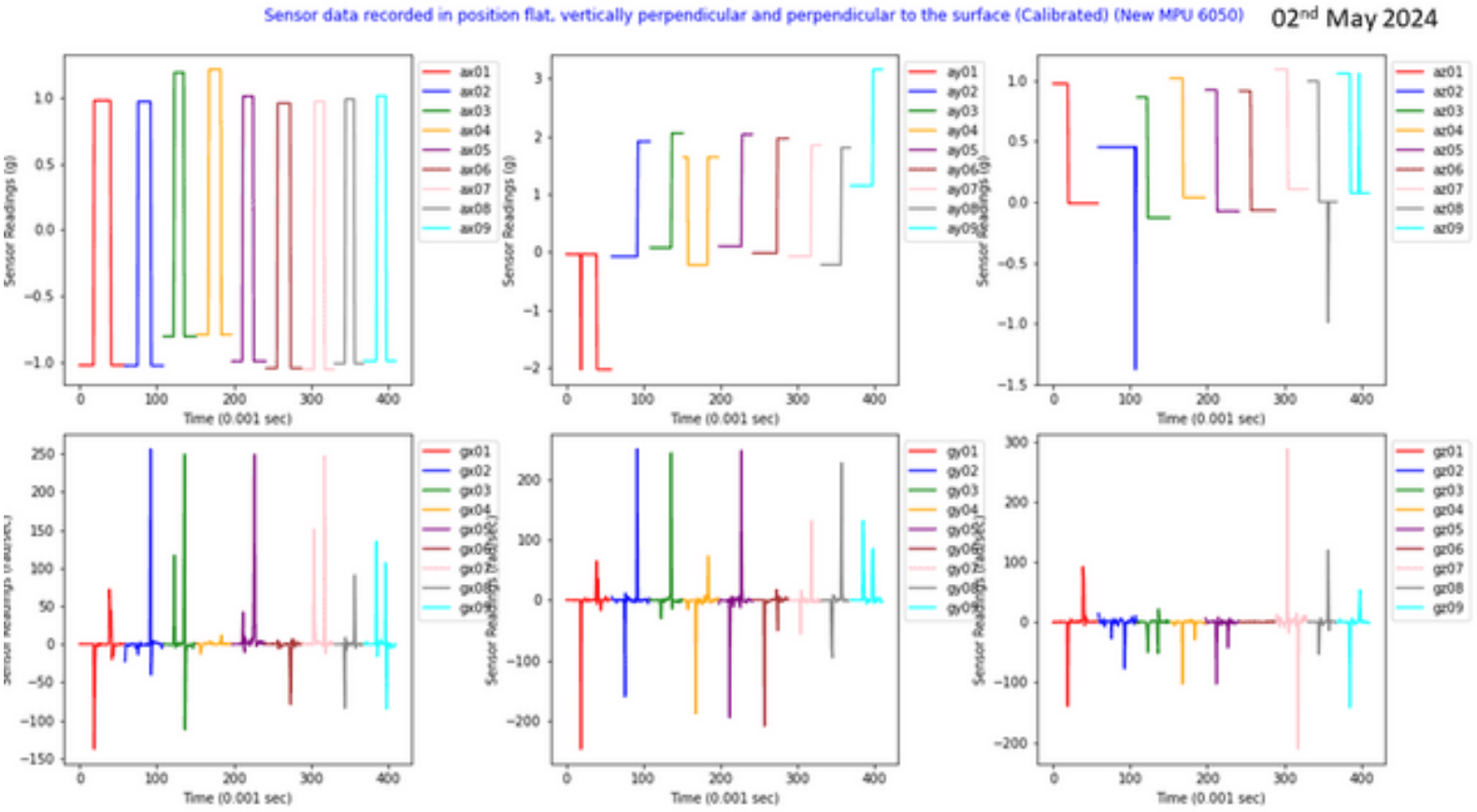


Figure 12 - Sensor analysis post-calibration showing improved accuracy.

Additionally, the offset is set. This data was captured at three distinct points to provide an accurate picture of the sensor's activity along all axes. The places were perpendicular to the surface both horizontally and vertically and flat on the surface.

3.3 Software Components

Considering the features and requirements of Raspberry Pi, MPU6050, and Machine Learning, the following software components were selected.

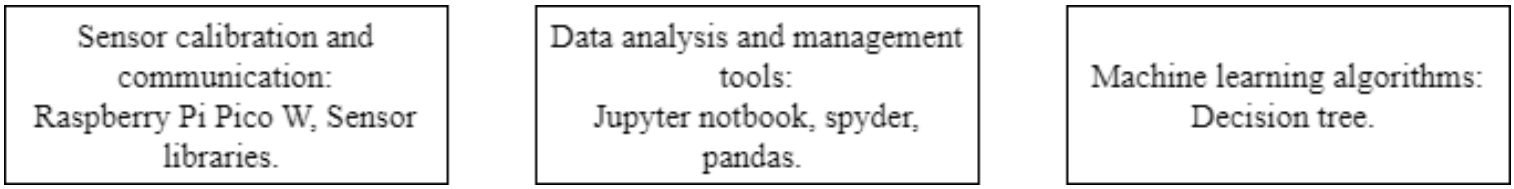


Figure 13 - Software Components used in this project.

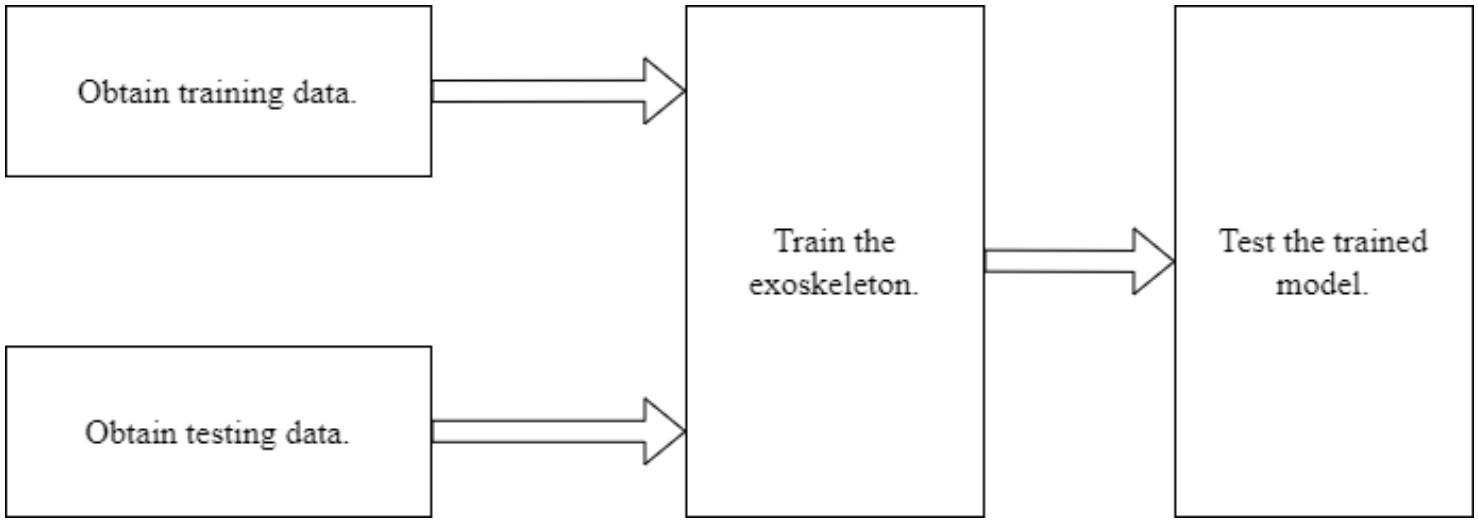


Figure 14 – Process flow of individual components in the exoskeleton system.

Figure 14 shows the steps for data preparation and training of the model with the dataset collected.

3.3.1 Mechanical Setup

The exoskeleton is made of 3D printing and a preexisting safety equipment, Dräger PSS Airboss [34]. There are two methods of 3D printing done on this exoskeleton. As mentioned in 2.1.3.1, the FDM method and SLS method. As shown in the figure below, the parts below the pelvic region are 3D printed using the FDM method, and all that above and around the waist region is 3D printed using the SLS method. The Airboss is then connected to these parts, making a complete exoskeleton.



Figure 15 - FDM method for 3D printing exoskeleton.



Figure 16 - SLS method for 3D printing exoskeleton.



Figure 17 - Exoskeleton with Airboss for enhanced functionality.

3.3.2 Experimental Setup

In this project, the exoskeleton was equipped with nine sensors placed at key points to capture accurate movement data. These sensors that are mounted on the exoskeleton measures linear acceleration and gyroscopic motion. Figure 18 shows the placement of the sensors on the exoskeleton, color coordinated for clarity.

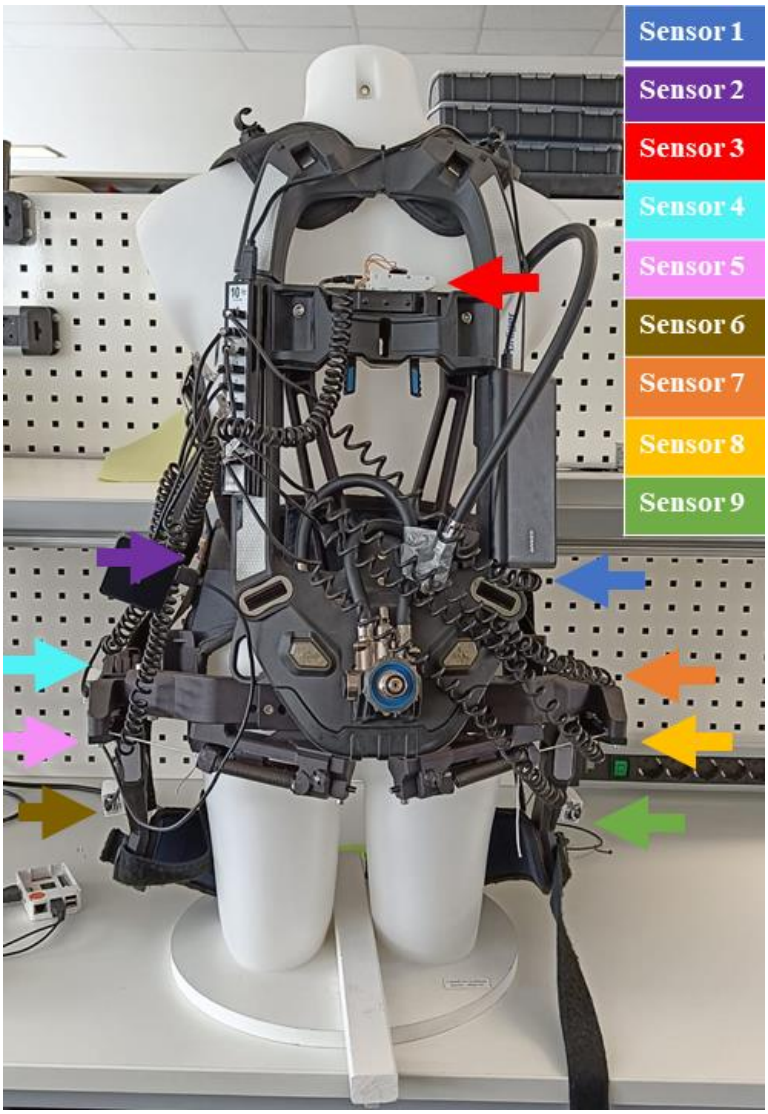


Figure 18 - Exact placements of 9 sensors on the exoskeleton (color coordinated)

Table 1 provides a description of each sensor and the movement components they measure:

Sensors	Measurement
Sensor 1	Helps in measuring the torsal bend and rotation.
Sensor 2	Helps in measuring the torsal bend and rotation.
Sensor 3	Mainly beneficial during forward bending and leigen bauch. It has a pressure sensor mounted on it for future purpose that involves altitude.
Sensor 4	Helps in measuring the rotation data during knee vor movement.
Sensor 5	Helps in measuring the rotation data during knee vor movement and dynamic movements.
Sensor 6	Helps in easily detecting walking or any dynamic movement.
Sensor 7	Helps in measuring the rotation data during knee vor movement.
Sensor 8	Helps in measuring the rotation data during knee vor movement and dynamic movements.
Sensor 9	Helps in easily detecting walking or any dynamic movement.

Table 1 - Placement and measurement description of the 9 sensors on the exoskeleton.

All nine sensors are paired with a Raspberry Pi Pico W and an MPU6050 module. Sensor 3, however, has an added pressure sensor, which is currently intended for future use. Activating this sensor during dynamic movement training, like when climbing stairs or moving across different altitudes, can offer significant advantages. It's especially beneficial for scenarios such as ascending or descending stairs in tall buildings, where pressure data can enhance the exoskeleton's ability to predict and respond to changing conditions.

The experimental configuration is uncomplicated. The Jupyter Notebook and Thonny include all the features needed to communicate and gather data. The data is transferred to the calibrated Raspberry Pi, and each Pico sends both the linear and gyroscopic acceleration values fixed on the exoskeleton.

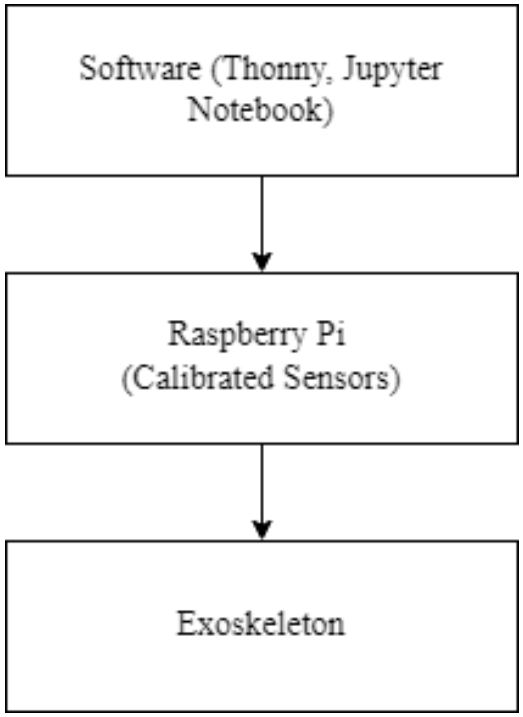


Figure 19 – Flowchart outlining the experiment process for exoskeleton testing.

The gyroscopic and linear acceleration data change with each movement of the exoskeleton and these differences are then recorded. The data recording procedure is depicted in the flowchart in Figure 19. The method that occurs in the software segment is illustrated in detail in Figure 20.

The parent script that nests the three scripts is called gui.py. At certain intervals, the Pico installed on the exoskeleton receives the tick from the sendTick.py script. Data from the exoskeleton Picos is fed into the livePrediction.py script. The trained exoskeleton analyzes the data received and displays the person's position, which is then published in the cloud.

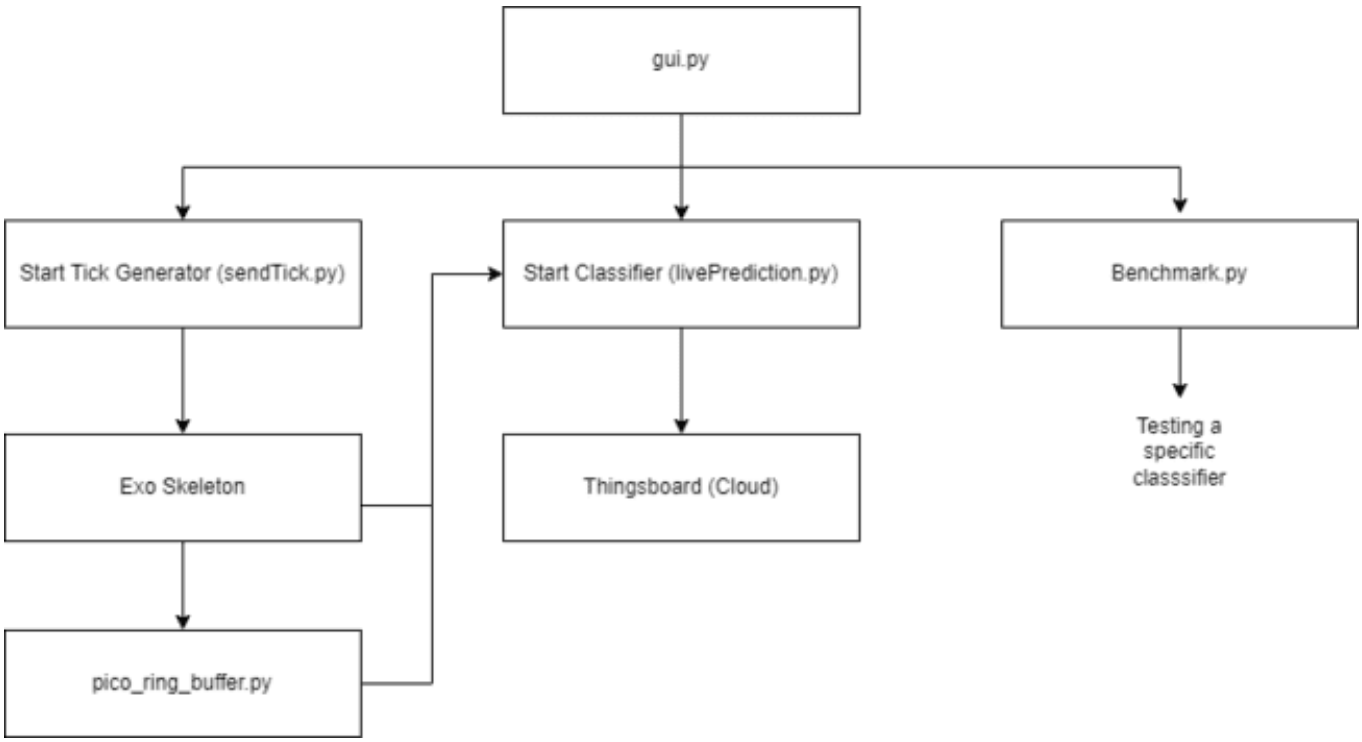


Figure 20 – Flowchart showcasing the software process involved in the exoskeleton system.

The benchmark.py script helps assess a specific classifier and benchmark the accuracy of the predictions.

3.3.3 Principle scheme

To understand how both software and hardware components function together during the experiments, the following visualization shown in Figure 21 explains the entire setup. The diagram shows how an exoskeleton equipped with sensors transmits the data wirelessly to the Raspberry Pi via a Wi-Fi router, which is then sent to the laptop for further analysis.

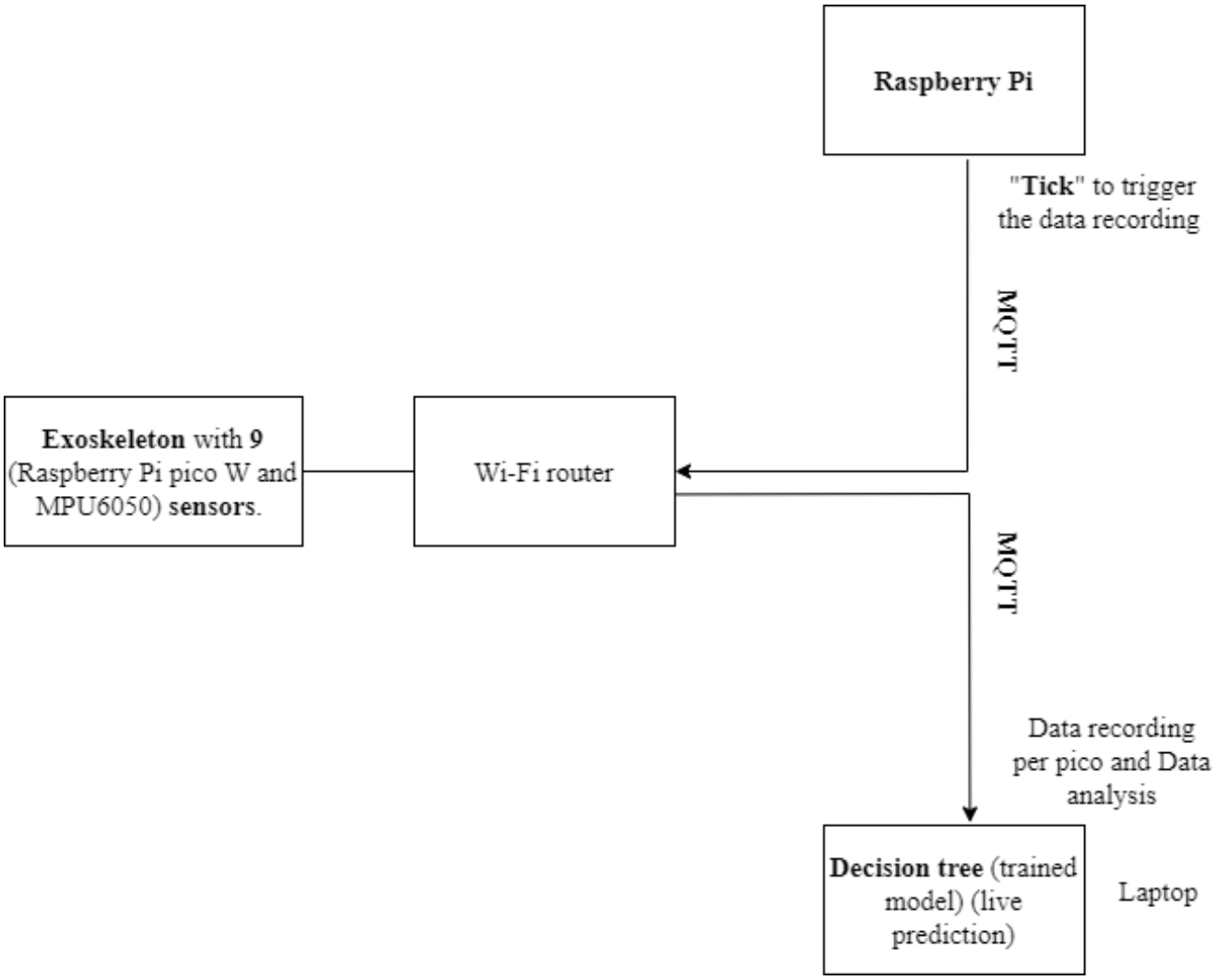


Figure 21 - Data Flow and System Architecture for Real-time Prediction in the Exoskeleton Setup

By presenting the entire workflow, this flowchart helps illustrate the seamless data flow from movement detection to live prediction, allowing for a more intuitive understanding of the experimental process.

3.3.4 Data Preparation

The dataset preparation is done in various positions. The wearer is asked to hold a static position for static training and do a few movements for dynamic training. The static data are collected in

standing, forward bending, balancing on the right knee, balancing on the left knee, and sleeping on the stomach. Figure 22 to Figure 26 gives a visual representation of the positions.



Figure 23 - Standing (Stehen)



Figure 22 - Forward Bending (Oberkoerper vor)



Figure 25 - Balance on right knee (knie links vor)

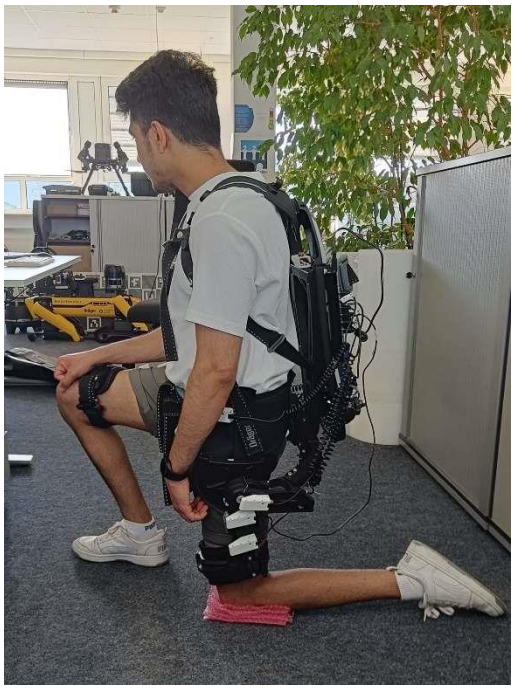


Figure 24 - Balance on left knee (knie rechts vor)

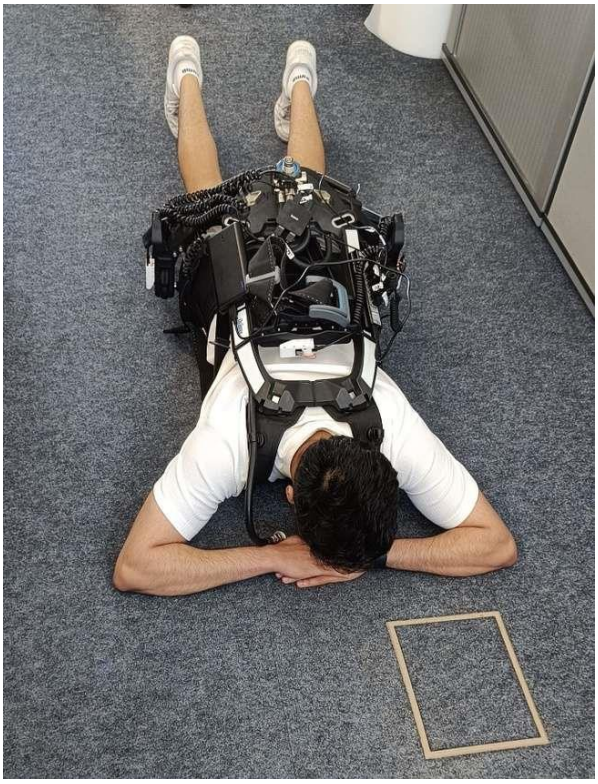


Figure 26 - Sleep on stomach (liegen bauch)

4 IMPLEMENTATION

This chapter provides a detailed explanation of the implementations conducted during the master's thesis. The first set completely explains the data preparation, the software involved, and the scripts. The second set explains in detail how to train the exoskeleton using Machine Learning.

4.1 Data Generation

The recording of data happens only after having a well-calibrated sensor. As explained in section 3.2, it is vital to have a precise calibration. Sensors have a sealed fate of manufacturing defects as they give vague values if not calibrated. The process of calibration takes place concerning the gravitation. The axes' calibration occurs in the direction of the gravitation, opposite to the gravitation, and perpendicular to the gravitation [35] [32]. Calibration is shown in Figure 27 to Figure 30 with respect to the axis. The calibration differs for the accelerometer and the gyroscopic part of the measurement. The accelerometer calibration is done as shown in Figure 27.

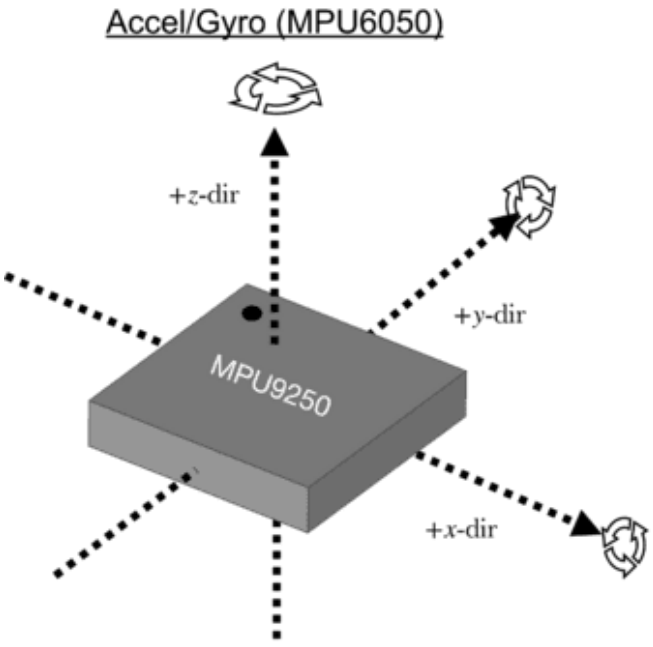


Figure 27 - Accelerometer calibration process to ensure accurate motion tracking [35].

The gyroscopic calibration is done as shown in Figure 28 to Figure 30.

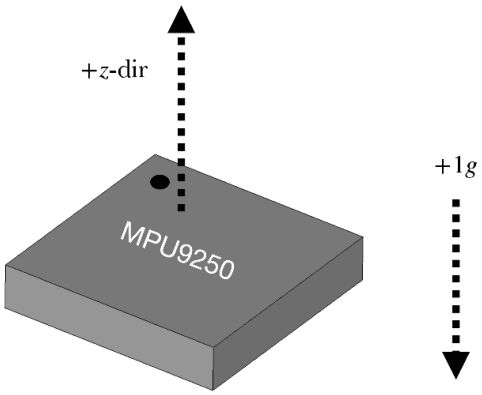


Figure 28 - Gyroscope calibration part 1: assessing rotational data accuracy [35].

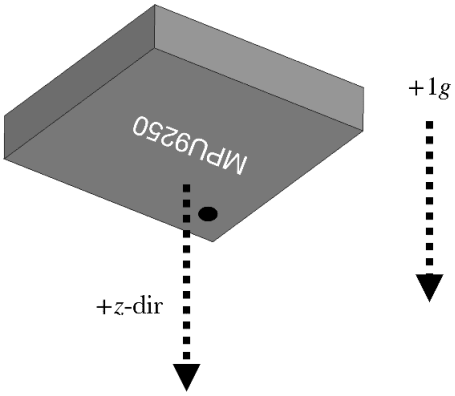


Figure 29 - Gyroscope calibration part 2: intermediate stage of data adjustment [35].

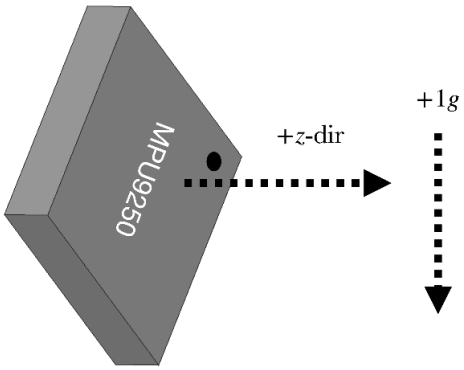


Figure 30 - Gyroscope calibration part 3: final calibration step for precise tracking [35].

A comprehensive understanding of the data generation process is essential to comprehend the project. The Pico emits gyroscopic value and linear acceleration, as indicated in Chapter 2.7. For more information, the Pico receives a constant tick at predetermined intervals. Changing the time interval to 100, 200, 300, 400, 500, and 1000 milliseconds is possible. A signal transmitted to the Raspberry Pi Pico over the MQTT topic is called a tick. The Pico uses the tick as a trigger to conduct specific tasks, including publishing sensor readings to an MQTT topic and broadcasting them. The Pico transmits the gyroscopic and acceleration measurements each time it receives the Tick. Once more, the MQTT topic gets these values. After that, these data are saved to a text file, for instance, Figure 31.

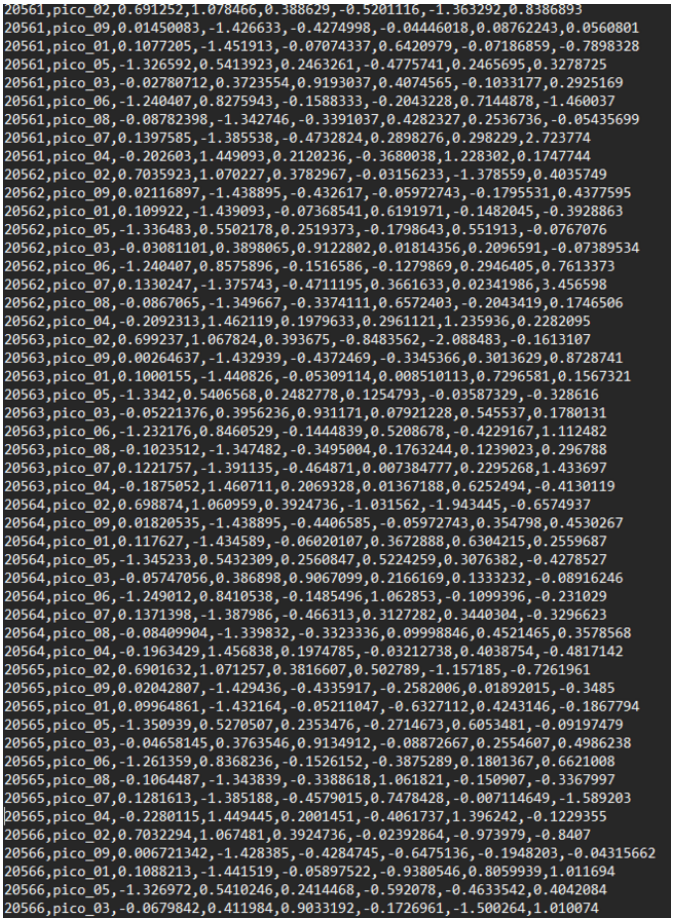


Figure 31 - Process of recording sensor data into a text file for analysis.

The data is recorded only when required and in CSV format, which is converted into a .txt file. The tick number and the six directions of freedom ax, ay, az, gx, gy, and gz values are recorded. This is sorted in ascending order according to the tick values. Once the data is sorted, it is stored in .csv format see Figure 32. Sorting it in ascending order makes parsing the data into its assigned column for all nine sensors easy.

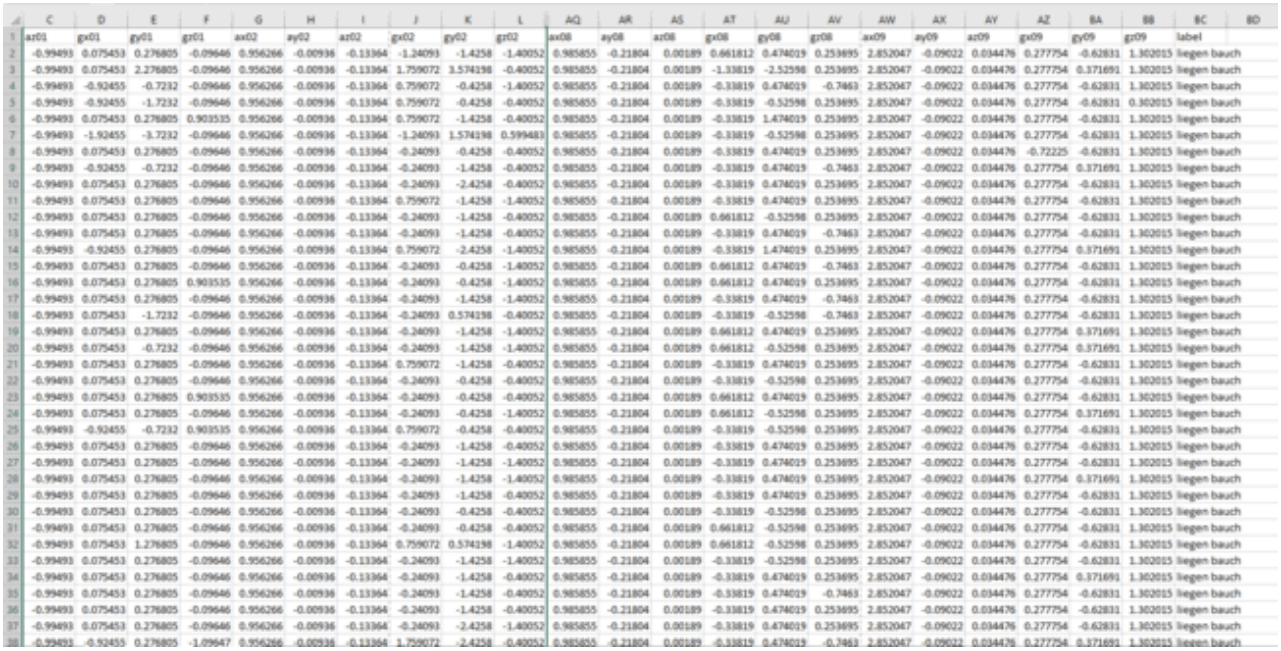
854230	pico_09	0.289376	-1.16983	-0.72552	-0.02919	-0.12612	0.376691
854230	pico_08	0.289513	-1.41087	0.002782	0.183958	0.062834	-0.17649
854230	pico_01	0.487468	-1.31817	0.202131	0.405457	-0.17111	0.523144
854230	pico_05	0.181418	1.448952	-0.07742	0.049143	0.185501	0.1523
854230	pico_03	-0.0004	1.210374	0.51194	0.086846	-0.14149	-0.59298
854230	pico_06	0.35194	1.335977	-0.58812	0.230792	0.096167	-1.69668
854230	pico_02	0.42158	1.299875	0.101487	-0.14607	-1.43963	-0.24528
854230	pico_04	0.894749	1.141065	-0.11815	-0.71152	0.762654	0.190042
854230	pico_07	1.038349	-1.14731	-0.06304	0.244026	-0.35063	-2.97852
854231	pico_09	0.296044	-1.17718	-0.71528	-0.18186	0.171592	0.040813
854231	pico_08	0.289885	-1.41597	0.00786	0.077088	0.093368	-0.03909
854231	pico_01	0.492238	-1.31817	0.196982	0.016144	0.141872	0.171999
854231	pico_05	0.18332	1.444539	-0.08742	-0.22567	0.238936	0.266804
854231	pico_03	-0.00678	1.222735	0.502011	0.231884	0.331797	-0.1426
854231	pico_06	0.368777	1.321364	-0.5929	0.055219	0.424412	-0.30736
854231	pico_02	0.421943	1.299189	0.111339	-0.23767	-1.65337	0.014262
854231	pico_04	0.894013	1.141065	-0.11888	-0.71915	0.724486	0.274011
854231	pico_07	1.016277	-1.15641	-0.0765	0.320362	-0.06818	-2.44416
854232	pico_09	0.296415	-1.17859	-0.72503	-0.04446	0.141058	-0.50117
854232	pico_08	0.27871	-1.41815	-0.00133	-0.06032	-0.04404	-0.69558
854232	pico_01	0.475727	-1.31852	0.192569	0.107747	-0.46882	-0.33182
854232	pico_05	0.196635	1.436817	-0.07376	-0.26383	0.551913	0.228636
854232	pico_03	-0.01166	1.212555	0.503222	1.208983	0.507369	0.132212
854232	pico_06	0.370273	1.327132	-0.57783	0.101021	0.699221	0.059047
854232	pico_02	0.42521	1.295413	0.106533	-0.26057	-1.43963	0.846323
854232	pico_04	0.895854	1.123815	-0.11064	-0.55885	1.02983	0.609889
854232	pico_07	1.034982	-1.13857	-0.06521	0.847079	-0.31246	0.578735
854233	pico_09	0.286413	-1.17894	-0.71894	-0.17423	-0.24062	-0.36377
854233	pico_08	0.281318	-1.41633	0.004475	-0.01452	-0.52495	-0.1994
854233	pico_01	0.484533	-1.31956	0.195756	-0.06783	-0.38485	0.30177
854233	pico_05	0.191309	1.438287	-0.08059	-0.29437	0.292371	0.304972
854233	pico_03	-0.00678	1.223826	0.498378	-0.1956	-0.02698	0.239082
854233	pico_06	0.376634	1.318672	-0.5819	-0.16615	0.432045	0.692635
854233	pico_02	0.426299	1.291294	0.106533	-0.05446	-1.21825	0.014262
854233	pico_04	0.904692	1.123815	-0.11791	0.21978	1.449677	0.426683
854233	pico_07	1.046954	-1.13682	-0.06761	0.022652	0.260061	-2.5205
854234	pico_09	0.293081	-1.17578	-0.71845	-0.0979	-0.33986	-0.08132

Figure 32 - Sorting collected data to facilitate easier analysis and visualization.

As mentioned above, sorted data helps in parsing; parsing is extracting relevant data from a larger set and organizing the data into a predefined format that is easy to read and understand. Since there are nine sensors, it is important and necessary to parse that data. The parsed data looks like the one shown in Figure 33. When the data is parsed in position, it is recorded in concatenates in the final column as a label, making the machine learning process easy.

It is important to have a robust dataset to conduct a clean and accurate analysis of the exoskeleton positions. Once all the data is clean and parsed, it is brought together to classify the positions using a decision tree. The decision tree is created after a few trials and errors in the number of branches that must be created. Changes are made in the in-depth range, random state, and test size.

The classification made using a decision tree is stored in the .pkl or pickle format, which can be used for live prediction in the future.



az01	gx01	gy01	gz01	ax02	ay02	az02	gx02	gy02	gz02	ax08	ay08	az08	gx08	gy08	gz08	ax09	ay09	az09	gx09	gy09	gz09	label
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-1.24093	-1.4258	-1.40052	0.985855	-0.21804	0.00189	0.661812	0.474019	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	2.276805	-0.09646	0.956266	-0.00936	-0.13364	1.759072	3.574198	-0.40052	0.985855	-0.21804	0.00189	-1.33819	-2.52598	0.253695	2.852047	-0.09022	0.034476	0.277754	0.371691	1.302015	liegen bauch
-0.99493	-0.92455	-0.7232	-0.09646	0.956266	-0.00936	-0.13364	0.759072	-0.4258	-1.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	-0.7463	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	-0.92455	-1.7232	-0.09646	0.956266	-0.00936	-0.13364	0.759072	-0.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	-0.52598	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	0.302015	liegen bauch
-0.99493	0.075453	0.276805	0.903535	0.956266	-0.00936	-0.13364	0.759072	-1.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	1.474019	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	-1.92455	-1.7232	-0.09646	0.956266	-0.00936	-0.13364	-1.24093	1.574198	0.599483	0.985855	-0.21804	0.00189	-0.33819	-0.52598	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-0.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	0.253695	2.852047	-0.09022	0.034476	-0.72225	-0.62831	1.302015	liegen bauch
-0.99493	-0.92455	-0.7232	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-1.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	-0.7463	2.852047	-0.09022	0.034476	0.277754	0.371691	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-2.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	0.759072	-1.4258	-1.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-1.4258	-0.40052	0.985855	-0.21804	0.00189	0.661812	-0.52598	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-1.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	-0.7463	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	-0.92455	0.276805	-0.09646	0.956266	-0.00936	-0.13364	0.759072	-2.4258	-1.40052	0.985855	-0.21804	0.00189	-0.33819	1.474019	0.253695	2.852047	-0.09022	0.034476	0.277754	0.371691	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-0.4258	-1.40052	0.985855	-0.21804	0.00189	0.661812	0.474019	-0.7463	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	0.903535	0.956266	-0.00936	-0.13364	-0.24093	-0.4258	-1.40052	0.985855	-0.21804	0.00189	0.661812	0.474019	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-1.4258	-1.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	-0.7463	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	-1.7232	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	0.574198	-0.40052	0.985855	-0.21804	0.00189	-0.33819	-0.52598	-0.7463	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-1.4258	-1.40052	0.985855	-0.21804	0.00189	0.661812	0.474019	0.253695	2.852047	-0.09022	0.034476	0.277754	0.371691	1.302015	liegen bauch
-0.99493	0.075453	-0.7232	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-1.4258	-1.40052	0.985855	-0.21804	0.00189	0.661812	-0.52598	0.253695	2.852047	-0.09022	0.034476	0.277754	0.371691	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	0.759072	-1.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-0.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	-0.52598	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	0.903535	0.956266	-0.00936	-0.13364	-0.24093	-0.4258	-0.40052	0.985855	-0.21804	0.00189	0.661812	0.474019	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-0.4258	-1.40052	0.985855	-0.21804	0.00189	0.661812	-0.52598	0.253695	2.852047	-0.09022	0.034476	0.277754	0.371691	1.302015	liegen bauch
-0.99493	-0.92455	-0.7232	0.903535	0.956266	-0.00936	-0.13364	0.759072	-0.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	-0.52598	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-1.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-1.4258	-1.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	0.253695	2.852047	-0.09022	0.034476	0.277754	0.371691	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-1.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-0.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	-0.52598	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-0.4258	-0.40052	0.985855	-0.21804	0.00189	0.661812	-0.52598	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	1.276805	-0.09646	0.956266	-0.00936	-0.13364	0.759072	0.574198	-1.40052	0.985855	-0.21804	0.00189	0.661812	-0.52598	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-1.4258	-1.40052	0.985855	-0.21804	0.00189	-0.33819	-0.52598	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-1.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	0.253695	2.852047	-0.09022	0.034476	0.277754	0.371691	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-1.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	-0.7463	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-1.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	0.075453	0.276805	-0.09646	0.956266	-0.00936	-0.13364	-0.24093	-2.4258	-1.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	0.253695	2.852047	-0.09022	0.034476	0.277754	-0.62831	1.302015	liegen bauch
-0.99493	-0.92455	0.276805	-1.09647	0.956266	-0.00936	-0.13364	1.759072	-2.4258	-0.40052	0.985855	-0.21804	0.00189	-0.33819	0.474019	-0.7463	2.852047	-0.09022	0.034476	0.277754	0.371691	1.302015	liegen bauch

Figure 33 - Parsed data displayed after processing for clearer interpretation.

4.2 Training the exoskeleton model

Once the data is collected, training the exoskeleton is next. Here, about two sets of data for each individual are collected for every position. The exoskeleton is trained using a decision tree, which is explained in 2.2.2. The classification of the data happens in the following method:

1. *Data preparation and handling:* The data is processed as described in 4.1 before being fed into the model. To prepare the data for training, all of the data files are concatenated into a single dataset using the pandas.concat() method. This guarantees that the model gets access to a wide range of data from various scenarios, which is critical for creating a more generalized exoskeleton model.
2. *Randomization and data splitting:* The dataset is divided into training and test sets to evaluate the model on a balanced and unknown fraction of the data. The split is done using the train_test_split() function. The training and test data sets can be split into different proportions, such as 80% to 20% or 70% to 30%, depending on the model's robustness. A random seed is used to verify that the splitting procedure is repeatable. This random seed ensures that, if the model is retrained, the data is split in the same way, resulting in consistency.
3. *Model selection and training:* The exoskeleton model is trained using the scikit-learn library's Decision Tree Classifier. Decision trees are chosen for their interpretability and ability to handle non-linear correlations between features, which is critical when working with complicated sensor data sets. The maximum depth of the decision tree ranges as

required. This iterative approach aids in determining the ideal depth of the tree while balancing complexity and performance. A shallow tree may underfit the data, whereas a deeper tree may cause overfitting. As a result, the model is trained for each depth, and the related accuracy is measured.

4. *Evaluating the model*: The model's accuracy is calculated by comparing its predictions to the actual test data. Models with 90% or higher accuracy are considered effective. Additionally, the importance of each feature is assessed to see which sensor readings have the most influence on the model's decisions.
5. *Saving the model*: Once a model performs well, it is saved using joblib so it can be reused without needing to retrain it.
6. *Iterating and tuning*: Different random seeds and tree depths are tested to fine-tune the model's performance. This ensures that the model generalizes well to various exoskeleton movements and avoids overfitting.

4.3 Live prediction

The implementation of the live prediction system to classify human activities based on sensor data is explained in detail. This system is designed to work in real-time, continuously gathering data, processing it, and predicting the user's pose using machine learning models. The live prediction system is built to classify activities based on data streamed from wearable sensors attached to various parts of the user's body. These sensors capture acceleration and gyroscopic data, which is transmitted via MQTT to a central processing unit for analysis.

The goal of this system is to ensure accurate and timely predictions, where predictions need to be made quickly enough to provide meaningful feedback in real time. The system not only predicts but also handles interruptions or incomplete data streams, ensuring robustness in a dynamic environment.

The system relies on MQTT to handle communication between the sensors and the prediction engine. Each sensor sends its data as soon as it is available. The data is transmitted in small packets, which include both the timestamp and the unique ID for each "tick" of data collection. The system receives the sensor data and organizes it in a buffer. It tracks which sensors have reported their data for a specific timestamp and checks if all required data (from all nine sensors) is present before attempting a prediction. This ensures that predictions are only made when complete data is available, reducing the chance of inaccuracies due to missing sensor readings.

To ensure efficiency, the system uses a ring buffer that dynamically stores the incoming data and automatically replaces old data. This keeps memory usage efficient while ensuring that predictions are based on recent and relevant information. Once the system has gathered a complete set of sensor data, it will proceed to make a prediction. The prediction process is as follows:

1. Data Preprocessing: The raw sensor data consists of six sensor readings per Pico. With nine Picos collecting data, a total of 54 features are used for each prediction (6 values * 9 Picos). This data is flattened and passed into the machine learning model for classification.
2. Prediction: The system uses a pre-trained machine learning model to predict the user's current activity. This model was trained using historical data and can classify activities.
3. Post-prediction Analysis: After the prediction is made, the system performs a rolling update to smooth out short-term fluctuations. A rolling buffer of the last few predictions is maintained, and the most common prediction in that buffer is selected as the final pose. This helps to filter out noise and ensures that the output is more stable, particularly when the user is in a static position for a period of time.

One of the challenges of working with real-time sensor data is ensuring the system can handle missing or delayed data without degrading performance. In this implementation, the system continuously checks whether all sensors have reported their data for each tick. If some data is missing, the system waits until it receives the necessary values or times out after a certain period, logging which sensors are missing. This approach prevents incomplete or inaccurate data from affecting predictions.

5 CHALLENGES AND MITIGATION STRATEGIES

This chapter outlines the key challenges encountered during the project and the strategies adopted to overcome them. Addressing these obstacles was crucial to ensure the success and robustness of the exoskeleton model.

5.1 Data Emission Signal

One of the first challenges was the inability to observe the data transmission process. It was unclear whether the Picos were receiving the tick signal and emitting data as expected. This uncertainty increased manual labor and slowed down progress, as missing signals made troubleshooting difficult.

To resolve this, an optical signal was added to the Picos, providing a visual indication of both data reception and emission. This simple solution allowed for real-time analysis and confirmation that the Picos were functioning correctly, significantly improving efficiency.

5.2 Data Loss

As the project progressed, random data loss at various tick IDs became a severe issue. Incomplete data sets made it impossible to train the model accurately, which led to incorrect classifications and faulty predictions. Figure 34 shows the possible location of the data loss. While manually fixing the missing data was an option, it was time-consuming and unreliable.

The solution was to modify the firmware configuration of the picos to implement block recording, capturing 30 measurements over 3 seconds per tick. Although this reduced the sampling frequency, it greatly improved the accuracy of the data for dynamic movements. This method gave the picos sufficient time to receive the tick via MQTT and publish the data for further processing. It also prevented data overlap and reduced the load on the system, ensuring smooth operation.

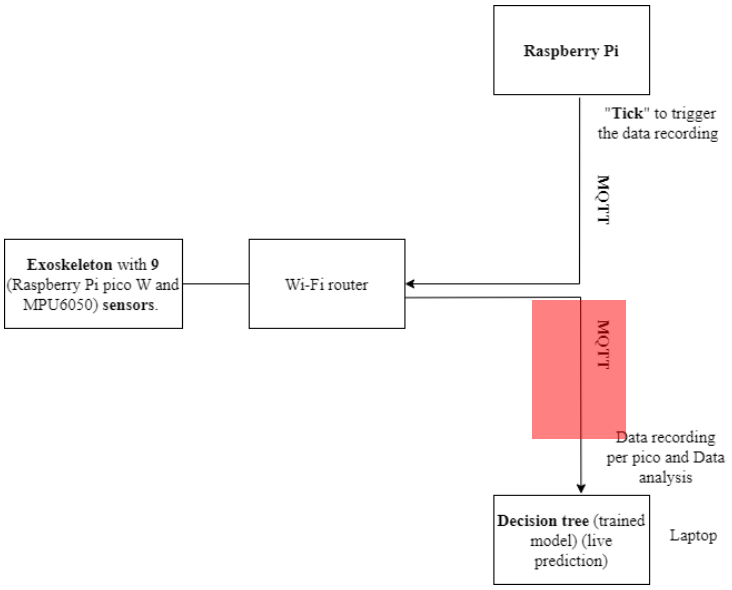


Figure 34 - Potential Location for Data Loss in the System.

5.3 Lost Pico Synchronization

Another significant challenge was the loss of synchronization between the picos. The data reception and emission became delayed, leading to data loss and disruptions in real-time model predictions. For the model to correctly predict the exoskeleton's position, it needed synchronized data from all nine sensors at once. Even though the block recording solution addressed data loss, it could not solve the real-time prediction issue caused by random emission delays.

To fix this, the implementation of Network Time Protocol (NTP) on the Raspberry Pi was crucial. Figure 35 shows the implementation of NTP. The tick script was modified to connect to an NTP server (pool.ntp.org), which synchronized the system's time with the current local time. This ensured that each pico emitted data with an accurate timestamp, allowing the model to sort the data chronologically. This synchronization made real-time predictions possible, improving the system's overall performance.

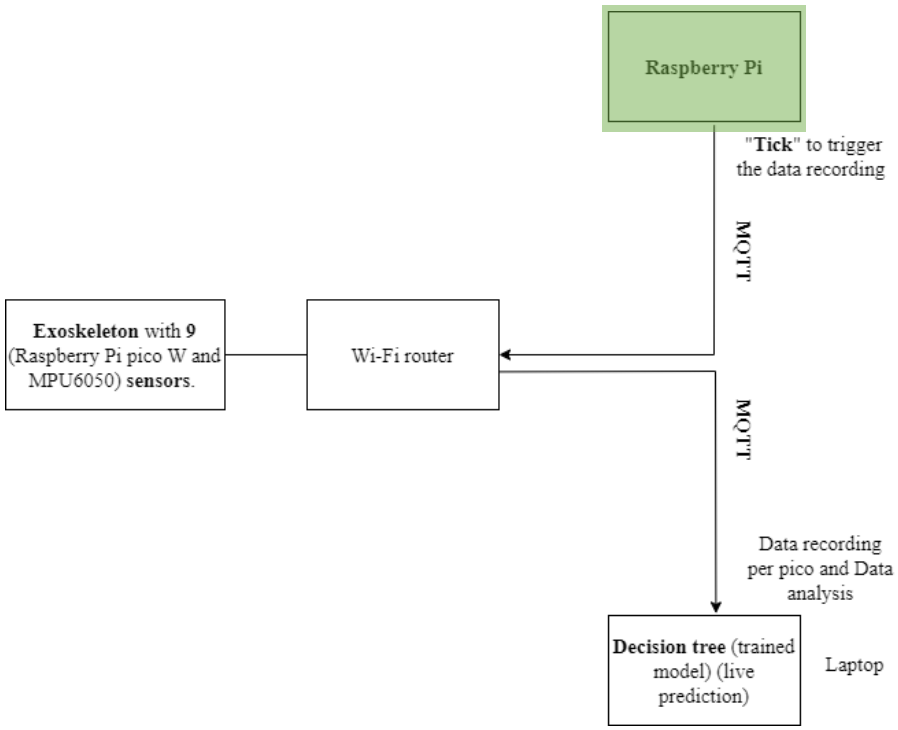


Figure 35 - Implementation of NTP in the system.

5.4 Classification Model

The decision tree model performed well for classifying static movements, as it could easily distinguish between different stationary positions. However, as the project evolved to include dynamic movements, the limitations of the decision tree became apparent. The model struggled to differentiate between similar actions like walking fast, walking slow, and standing, resulting in lower accuracy.

One of the key issues was that the decision tree lacked the ability to incorporate a time factor, which is essential for dynamic movement classification. To address this limitation and improve the model's robustness, alternative methods like Random Forest, sliding window approaches, or more advanced models such as K-Nearest Neighbors (KNN) or 3D Convolutional Neural Networks (3D-CNN) could be explored. Although these models may be more complex to train, they offer the potential for greater accuracy and improved performance for dynamic movements. These could be the next steps for future iterations of the project to achieve more reliable results.

6 RESULTS

This section presents the analysis of static and dynamic movements under various positions and conditions.

6.1 Static Data analysis

The static movement under various positions and conditions, particularly standing, forward bending, balance on the left knee, balance on the right knee, and sleeping on the stomach, was analyzed. The exoskeleton's live prediction system successfully detects the wearer's static positions in real time. Using accelerometers and gyroscopes, the exoskeleton can identify various static postures with high accuracy. The key results are presented below.

- **System Overview**

The exoskeleton’s live prediction system operates by continuously monitoring the sensor readings. These readings are processed through a machine learning algorithm trained to recognize static positions.

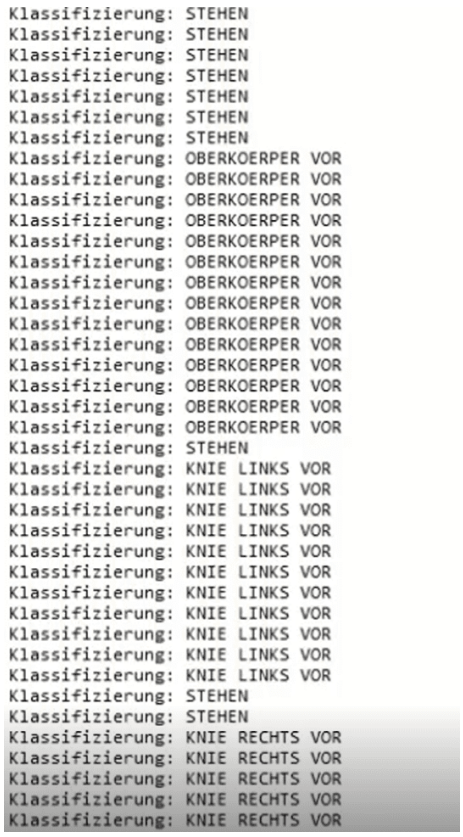


Figure 36 - Data flow diagram for live prediction of movement using sensors.

Figure 36 illustrates the data flow, showing how the real-time sensor input is mapped to predefined static positions. The prediction model works with a latency of less than 50 ms, ensuring immediate feedback to the user.

- **Accuracy and Performance**

During testing, the system achieved a prediction accuracy of 98%, meaning that nearly all static positions were correctly identified. As shown in Table 2, the system exhibited strong performance across different static postures:

Static Position	Accuracy	Latency (ms)
Standing	98%	14 ms
Forward Bending	95%	25.7 ms
Balance on the left knee	95%	22.3 ms
Balance on the right knee	95%	22.3 ms
Sleeping on stomach	97%	32.5 ms

Table 2 – Live prediction performance.

The system's prediction was most accurate in standing positions while sitting and bending positions exhibited slightly lower but still acceptable accuracy levels. False positives were minimal, indicating that the system rarely predicted a static position when none was present.

- **Visual Representation of Static Prediction**

Instead of a video demonstration, Figure 37 and Figure 38show snapshots of the live prediction system at different stages. These images capture key moments where the exoskeleton correctly identifies static positions:

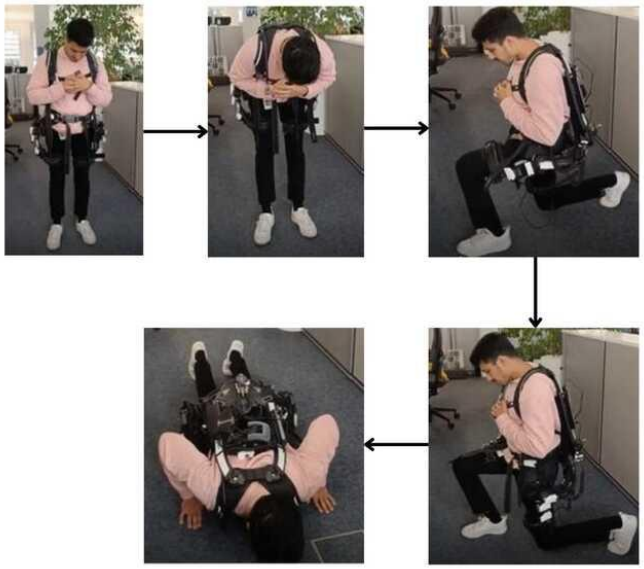


Figure 37 - Transition between static positions during movement detection analysis.

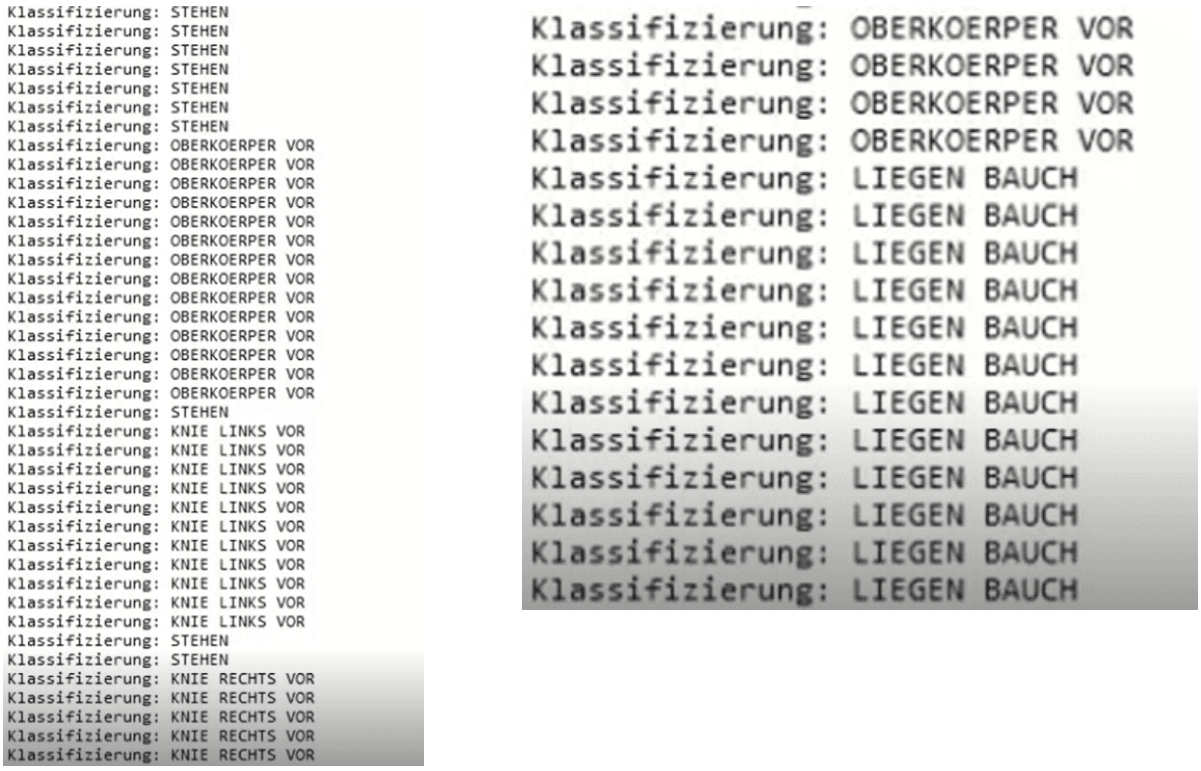


Figure 38 - Live detection of body positions captured by the exoskeleton sensors.

These figures demonstrate the exoskeleton's ability to provide reliable feedback in real-time.

- **Discussion**

The live prediction system has proven effective at identifying static positions with minimal latency. Although a video demonstration would more clearly show this process, the results presented here provide strong evidence that the system is capable of delivering real-time feedback based on static postures. This capability is critical for ensuring the wearer's safety and optimizing movement assistance.

6.2 Dynamic data analysis

The analysis of dynamic movement under various walking conditions, specifically Gehen Schnell (fast walking) and Gehen Langsam (slow walking). The data have been analyzed using both the Fast Fourier Transform (FFT) for frequency-domain insights and time-domain analysis to understand sensor readings.

- **Fast Walking (Gehen Schnell)**

Frequency Domain:

The FFT analysis shows clear peaks across the frequency spectrum, particularly around 0.1429 Hz. This consistent peak, visible in both the accelerometer (axes ax09, ay06) and gyroscope (axes gx09, gy06, gz06) readings, indicates periodic motion associated with the walking pace. The magnitude of these peaks suggests a high level of activity in the 0.1 to 0.2 Hz range, corresponding to fast steps. The variance in magnitudes across different axes shows differing dynamics for lateral and vertical movements during walking.

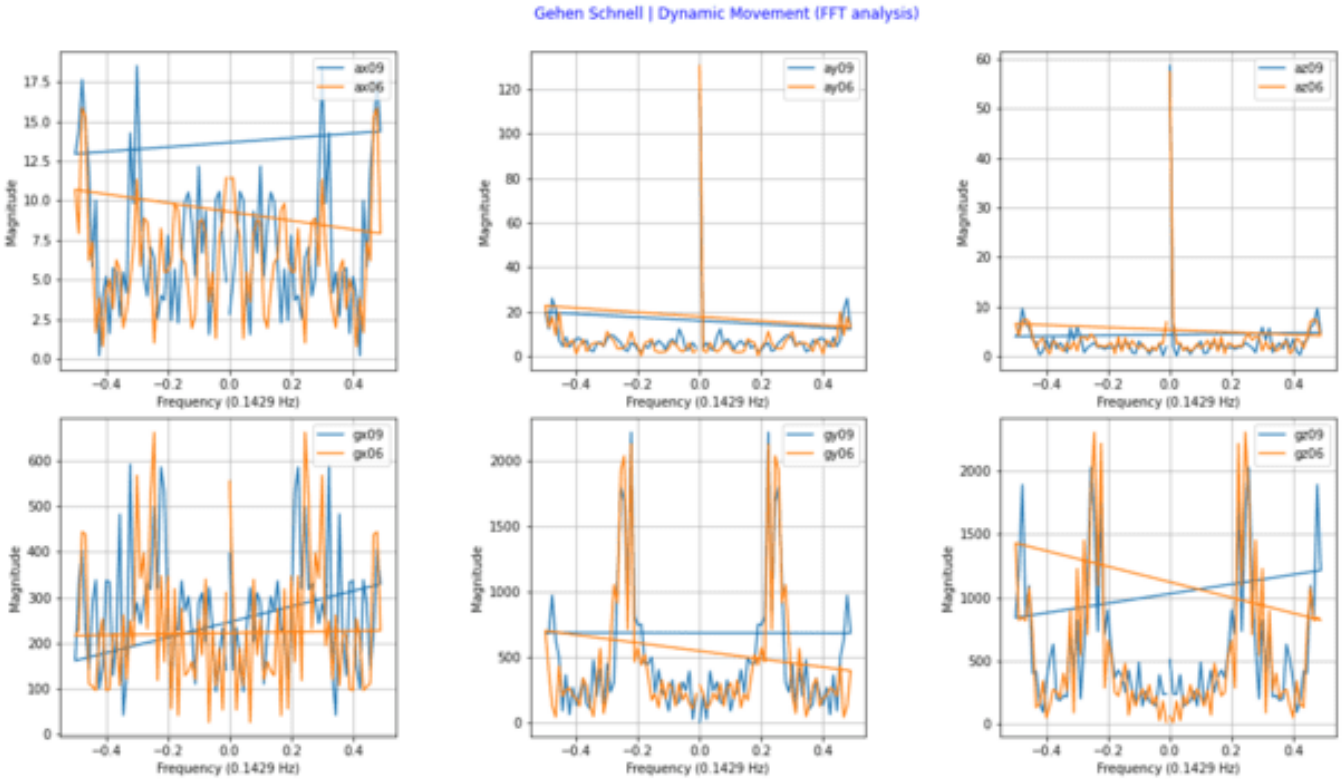


Figure 39 - Fast walking movement frequency spectrum (FFT analysis).

Time Domain:

The time-domain analysis reveals fluctuating sensor readings in response to the fast-walking pace. The accelerometer data (ax09, ax06) shows periodic peaks in the y-axis, which corresponds to the repetitive impact of footfalls. In contrast, the gyroscope readings (gx06, gy06, gz06, gx09, gy09, gz09) show consistent oscillations, suggesting active rotational movements as the body shifts between steps.

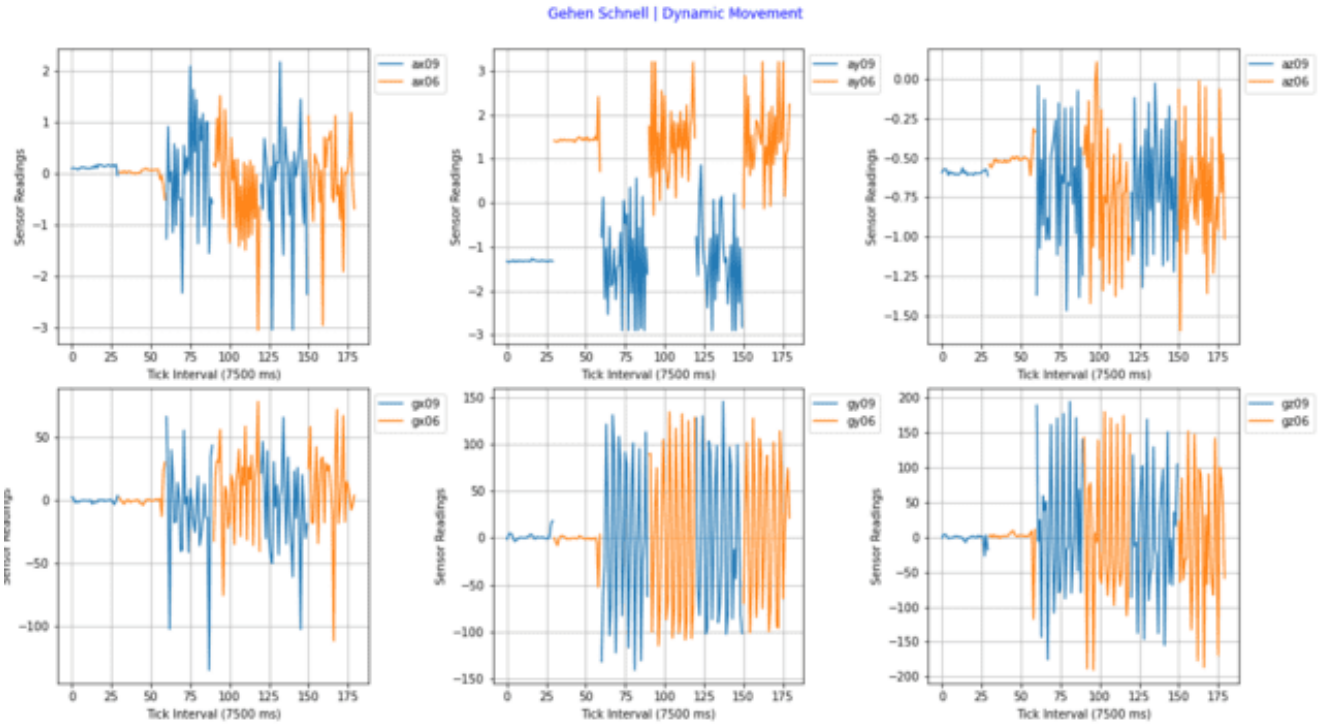


Figure 40 - Time-domain representation of fast walking movement data.

- **Slow Walking (Gehen Langsam)**

Frequency Domain:

The frequency-domain analysis for slow walking displays similar peaks at around 0.1429 Hz but with lower magnitudes compared to fast walking. This lower magnitude indicates a reduction in overall energy exertion and movement frequency. The peaks are less pronounced, suggesting smoother and more controlled movements, which are typical of a slower walking pace.

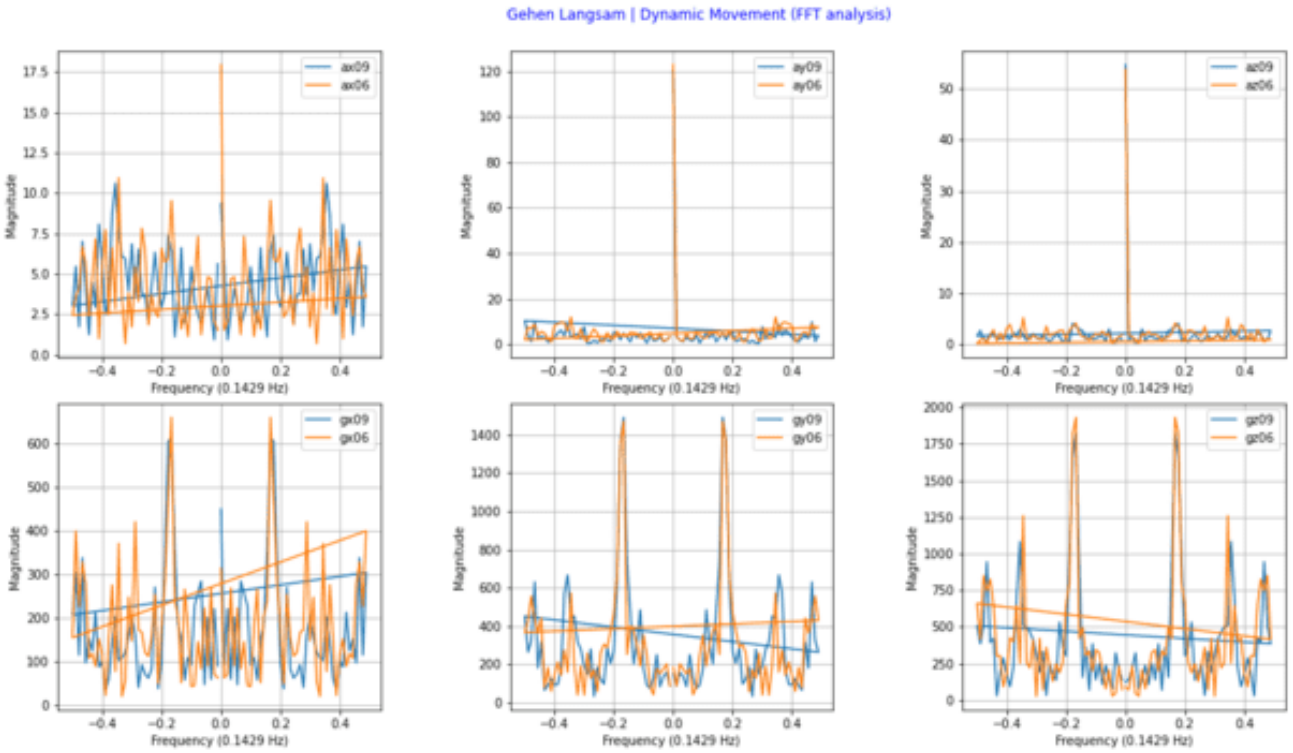


Figure 41 - Slow walking movement frequency spectrum (FFT analysis).

Time Domain:

The time-domain sensor readings for slow walking show a more spread-out pattern with less frequent oscillations in both the accelerometer (ax09, ay06) and gyroscope (gx09, gz06) data. The movement patterns appear steadier, with smaller peaks and reduced rotational shifts, indicating a more stable walking style when compared to dynamic, fast walking.

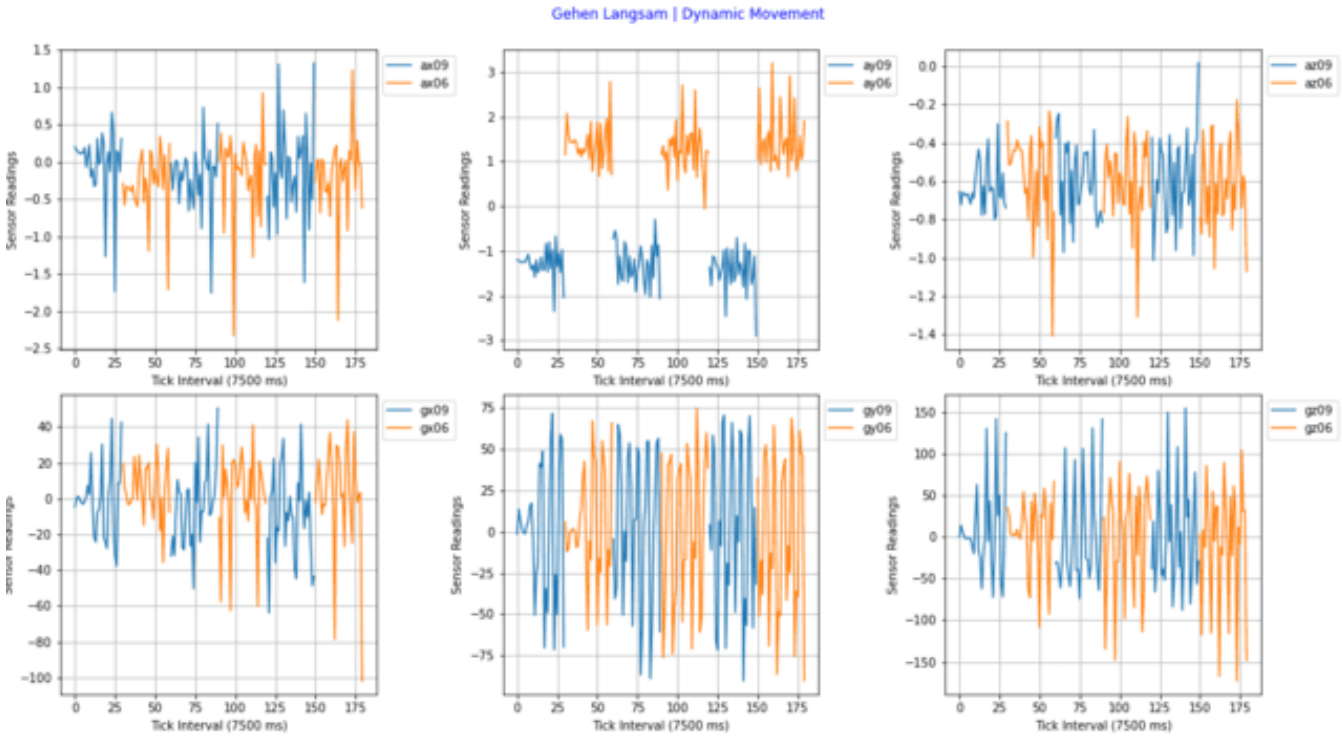


Figure 42 - Time-domain representation of slow walking movement data.

- **Comparative Analysis**

When comparing fast and slow walking, we observe clear differences in the magnitude and frequency of movements captured by the sensors. The frequency domain highlights the energy exertion differences, with higher magnitudes in fast walking. Similarly, the time-domain analysis illustrates the distinction in step frequency and body rotation between the two walking speeds.

7 CONCLUSION AND OUTLOOK

This chapter gives an overview of the final thesis. It discusses the main function and the goals achieved, as well as an outlook on future improvements and enhancements.

7.1 Conclusion

Machine learning has been playing a significant role in making the model more convenient. In the context of this master's thesis, the extent to which the model can be trained and how this model can minimize human effort in a hazardous situation was analyzed. The primary objective of this thesis was to develop a smart exoskeleton integrated with machine learning algorithms to enhance safety and movement detection in real-time mainly of the firefighters.

Throughout this research, the exoskeleton successfully detects the static movement with high accuracy. The integration of machine learning modules, particularly decision tree, enabled the exoskeleton to adaptively augment human movements and, hence providing the real-time detection. The static detection accuracy reached up to 98%, while dynamic movements data were collected and analyzed but unfortunately the limitations of decision trees became apparent. Using random forest was also not that successful. The dynamic movement detection needs a high-end machine learning model training. On the contrary, these data were analyzed with detailed frequency domain and time domain metrics. These results underline the proper working of the sensors and a hope of improvement for future developments.

This project was not without its challenges. Early on, the data loss and synchronization issues posed significant obstacle for accurate predictions. By implementing Network Time Protocol (NTP) and block recording, I successfully mitigated the issues, ensuring synchronization across the sensors and also accurate prediction. These modifications not only fixed the problem but also improved the robustness of the system and the reliability of the real-time predictions. Such solutions were crucial to know and push the boundaries of the exoskeleton to what it can achieve.

The research conducted here contributes to the greatest part of future development of the wearable technology. It is demonstrating how machine learning can significantly enhance the practical implementation and enhance the performance of the exoskeleton. Beyond industrial application, this technology benefits the medical field, military, and elderly care. By implementing real-time prediction and feedback, this system has paved a way for more advancements in the field of exoskeleton.

While the result of this thesis is promising, there remains ample room for improvement. One potential room of improvement can be use of more advanced machine learning models which increases the robustness and complexity of the system. Additionally, increasing the exoskeleton’s ability to recognize and adapt to new and more complex movements. Lastly, broadening the application of this exoskeleton to broader adoption and more diverse functionalities making it an invaluable tool for physical assistance.

In conclusion, the development of a smart exoskeleton represents a significant advancement in wearable technology. By merging exoskeleton with machine learning modules, this project lays a bigger groundwork for a much bigger advancement in the future. This is initial phase of a revolution in the industry of safety and physical assistance.

7.2 Outlook

Currently, as part of this final thesis, the detection of static and dynamic movements has made significant strides, setting the stage for future advancements. While challenges like hardware integration and the need for extensive datasets were encountered, overcoming these obstacles has laid a solid foundation for future studies. These insights pave the way for exploring more advanced machine learning models and expanding datasets to enhance the system’s robustness, efficiency, and reliability.

Looking ahead, the field of smart exoskeletons holds tremendous potential for innovation across various sectors. The following future directions can help build on the groundwork established by this thesis:

- Enhanced Machine Learning Models: Exploring more sophisticated algorithms, such as deep learning, could significantly improve the exoskeleton's accuracy and responsiveness. Implementing real-time learning capabilities would allow the system to adapt dynamically to the user's movements and environments.
- Broader Application Scope: Beyond industrial use, smart exoskeletons could be extended to sectors like healthcare, rehabilitation, and military applications. Tailoring the technology to assist with specific tasks or medical conditions would significantly increase its utility and adoption in diverse fields.
- Improved User Interface and Experience: Developing more intuitive user interfaces and feedback mechanisms would enhance the overall user experience. A user-friendly app could make interactions with the exoskeleton more immersive, providing valuable insights and enhancing ease of use.

In conclusion, the successful completion of this thesis marks an important milestone in the development of smart exoskeletons for safety applications. However, it also opens up numerous opportunities for further research and development. As technology continues to evolve, the integration of machine learning with exoskeletons will undoubtedly play a key role in shaping the future of workplace safety, healthcare, and more.

8 REFERENCE

[1] O. Coser, C. Tamantini, P. Soda, and L. Zollo, "AI-based methodologies for exoskeleton-assisted rehabilitation of the lower limb: a review," *Front. Robot. AI*, vol. 11, p. 1341580, Feb. 2024, doi: 10.3389/frobt.2024.1341580.

[2] S. Yu *et al.*, "Design and Control of a High-Torque and Highly Backdrivable Hybrid Soft Exoskeleton for Knee Injury Prevention During Squatting," *IEEE Robot. Autom. Lett.*, vol. 4, no. 4, pp. 4579–4586, Oct. 2019, doi: 10.1109/LRA.2019.2931427.

[3] E. Jovanov, A. Milenkovic, C. Otto, and P. C. de Groen, "A wireless body area network of intelligent motion sensors for computer assisted physical rehabilitation," *J. NeuroEngineering Rehabil.*, vol. 2, no. 1, p. 6, Mar. 2005, doi: 10.1186/1743-0003-2-6.

[4] D. Ferris, "The exoskeletons are here," *J. Neuroengineering Rehabil.*, vol. 6, p. 17, Feb. 2009, doi: 10.1186/1743-0003-6-17.

[5] O. Sandoval-Gonzalez *et al.*, "Design and Development of a Hand Exoskeleton Robot for Active and Passive Rehabilitation," *Int. J. Adv. Robot. Syst.*, vol. 13, no. 2, p. 66, Mar. 2016, doi: 10.5772/62404.

[6] P. Stegall, K. Winfree, D. Zanotto, and S. K. Agrawal, "Rehabilitation Exoskeleton Design: Exploring the Effect of the Anterior Lunge Degree of Freedom," *IEEE Trans. Robot.*, vol. 29, no. 4, pp. 838–846, Aug. 2013, doi: 10.1109/TRO.2013.2256309.

[7] J. Kuschan *et al.*, "PowerGrasp - Design and Evaluation of a Modular Soft-Robotic Arm Exosuit for Industrial Applications".

[8] K. Huysamen, M. de Looze, T. Bosch, J. Ortiz, S. Toxiri, and L. W. O'Sullivan, "Assessment of an active industrial exoskeleton to aid dynamic lifting and lowering manual handling tasks," *Appl. Ergon.*, vol. 68, pp. 125–131, Apr. 2018, doi: 10.1016/j.apergo.2017.11.004.

[9] "German_Whitepaper_EN_Digital-2023 (1).pdf."

[10] "OSullivan_2016_exoskeletons.pdf."

[11] X. Wang, M. Jiang, Z. Zhou, J. Gou, and D. Hui, "3D printing of polymer matrix composites: A review and prospective," *Compos. Part B Eng.*, vol. 110, pp. 442–458, Feb. 2017, doi: 10.1016/j.compositesb.2016.11.034.

[12] G. N. Levy, R. Schindel, and J. P. Kruth, "RAPID MANUFACTURING AND RAPID TOOLING WITH LAYER MANUFACTURING (LM) TECHNOLOGIES, STATE OF THE ART AND FUTURE PERSPECTIVES," *CIRP Ann.*, vol. 52, no. 2, pp. 589–609, 2003, doi: 10.1016/S0007-8506(07)60206-6.

[13] N. Shahrubudin, T. C. Lee, and R. Ramlan, "An Overview on 3D Printing Technology: Technological, Materials, and Applications," *Procedia Manuf.*, vol. 35, pp. 1286–1296, 2019, doi: 10.1016/j.promfg.2019.06.089.

[14] Ö. Keleş, C. W. Blevins, and K. J. Bowman, "Effect of build orientation on the mechanical reliability of 3D printed ABS," *Rapid Prototyp. J.*, vol. 23, no. 2, pp. 320–328, Mar. 2017, doi: 10.1108/RPJ-09-2015-0122.

[15] I. H. Sarker, "Machine Learning: Algorithms, Real-World Applications and Research Directions," *SN Comput. Sci.*, vol. 2, no. 3, p. 160, Mar. 2021, doi: 10.1007/s42979-021-00592-x.

[16] W. Jin, "Research on Machine Learning and Its Algorithms and Development," *J. Phys. Conf. Ser.*, vol. 1544, p. 012003, May 2020, doi: 10.1088/1742-6596/1544/1/012003.

[17] B. Mahesh, "Machine Learning Algorithms - A Review," vol. 9, no. 1, 2018.

[18] "eBook: Machine Learning - kurz & gut von Chi Nhan Nguyen | ISBN 978-3-96010-511-4 | Sofort-Download kaufen - Lehmanns.de." Accessed: Apr. 25, 2024. [Online]. Available: https://www.lehmanns.de/shop/mathematik-informatik/56734564-9783960105114-machine-learning-kurz-amp-gut

[19] H. Zhang, L. Zhang, and Y. Jiang, "Overfitting and Underfitting Analysis for Deep Learning Based End-to-end Communication Systems," in *2019 11th International Conference on Wireless Communications and Signal Processing (WCSP)*, Oct. 2019, pp. 1–6. doi: 10.1109/WCSP.2019.8927876.

[20] A. Bhande, "What is underfitting and overfitting in machine learning and how to deal with it.," GreyAtom. Accessed: May 07, 2024. [Online]. Available: https://medium.com/greyatom/what-is-underfitting-and-overfitting-in-machine-learning-and-how-to-deal-with-it-6803a989c76

[21] M. A. Khan *et al.*, "A Deep Learning-Based Intrusion Detection System for MQTT Enabled IoT," *Sensors*, vol. 21, no. 21, p. 7016, Oct. 2021, doi: 10.3390/s21217016.

[22] M. V. Masdani and D. Darlis, "A comprehensive study on MQTT as a low power protocol for internet of things application," *IOP Conf. Ser. Mater. Sci. Eng.*, vol. 434, p. 012274, Dec. 2018, doi: 10.1088/1757-899X/434/1/012274.

[23] P. Di Felice and G. Paolone, "Papers Mentioning Things Board: A Systematic Mapping Study," *J. Comput. Sci.*, vol. 20, no. 5, pp. 574–584, May 2024, doi: 10.3844/jcssp.2024.574.584.

[24] thingsboard, "What is ThingsBoard?," ThingsBoard. Accessed: Sep. 13, 2024. [Online]. Available: https://thingsboard.io/docs/getting-started-guides/what-is-thingsboard/

[25] thingsboard, "Guides," ThingsBoard. Accessed: Sep. 13, 2024. [Online]. Available: https://thingsboard.io/docs/guides/

[26] L. O. Aghenta, M. Tariq Iqbal, and Department of Electrical and Computer Engineering, Faculty of Engineering and Applied Science, Memorial University of Newfoundland (MUN), St. John's, NL A1B 3X5, Canada, "Design and implementation of a low-cost, open source IoT-based SCADA system using ESP32 with OLED, ThingsBoard and MQTT protocol," *AIMS Electron. Electr. Eng.*, vol. 4, no. 1, pp. 57–86, 2020, doi: 10.3934/ElectrEng.2020.1.57.

[27] X. Jie, X. Liang, D. Lian, and Z. Delin, "Research on network timing system based on NTP," in *2017 13th IEEE International Conference on Electronic Measurement & Instruments (ICEMI)*, Yangzhou, China: IEEE, Oct. 2017, pp. 356–360. doi: 10.1109/ICEMI.2017.8265967.

[28] S. Rangaswamy, "Department of Computer Science and Engineering R V College of Engineering Bengaluru, India," vol. 27, no. 6, 2021.

[29] "Raspberry Pi Models and Features," Spiceworks Inc. Accessed: May 08, 2024. [Online]. Available: https://www.spiceworks.com/tech/networking/articles/what-is-raspberry-pi/

[30] R. P. Ltd, "Buy a Raspberry Pi Pico," Raspberry Pi. Accessed: May 08, 2024. [Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-pico/

[31] T. Jaiswal, "MPU6050 Setup and Calibration Guide," Instructables. Accessed: Apr. 24, 2024. [Online]. Available: https://www.instructables.com/MPU6050-Setup-and-Calibration-Guide/

[32] "Accelerometer & Gyroscope Module (MPU6050) — SunFounder Ultimate Sensor Kit documentation." Accessed: May 08, 2024. [Online]. Available: https://docs.sunfounder.com/projects/ultimate-sensor-kit/en/latest/components_basic/05-component_mpu6050.html

[33] "Arceli GY-521 MPU6050 3 Axis Accelerometer Gyroscope Module 6 DOF 6-Axis Acceleration Sensor Gyroscope Sensor Module 16 Bit AD Converter Data Output IIC for Arduino: Amazon.de: Business, Industry & Science." Accessed: May 08, 2024. [Online]. Available: https://www.amazon.de/-/en/MPU6050-Accelerometer-Gyroscope-Acceleration-Converter/dp/B07BVXN2GP

[34] D. Pss, "Dräger PSS AirBoss".

[35] "Gyroscope and Accelerometer Calibration with Raspberry Pi," Maker Portal. Accessed: Apr. 24, 2024. [Online]. Available: https://makersportal.com/blog/calibration-of-an-inertial-measurement-unit-imu-with-raspberry-pi-part-ii

9 APPENDIX

Libraries	Description
NumPy	It provides support for large, multidimensional arrays and matrices and a collection of mathematical functions for operating on them.
Matplotlib	A plotting library used for creating static, animated, and interactive visualizations in Python.
Pandas	It offers data manipulation and analysis tools, particularly for structured data, and provides data frames similar to those in R.
Scikit-learn	A machine learning library that provides simple and efficient tools for data mining and data analysis, built on NumPy, SciPy, and Matplotlib.
OpenCV	A library for computer vision tasks that provides tools for image processing and real-time video analysis.
GPIO Zero	A library designed for controlling the GPIO pins on Raspberry Pi, simplifying the process of connecting and interacting with sensors and devices.
paho.mqtt.client	A client library for the MQTT protocol used for lightweight messaging and communication.
imu	A library for interfacing with the MPU6050 Inertial Measurement Unit (IMU) to read acceleration and gyroscope data.
machine	MicroPython module that allows access to hardware components and peripherals on microcontroller boards.